



UNIVERSITÀ
di **VERONA**

Deep Learning

Multimodal Foundation Models

Cigdem Beyan

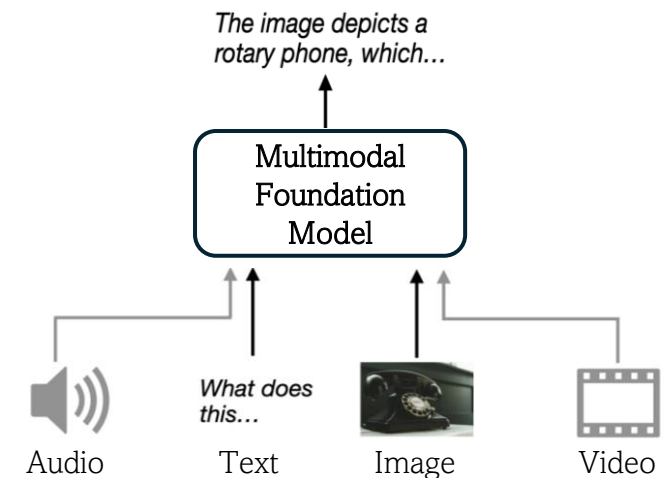
A.Y. 2025-2026

TODAY

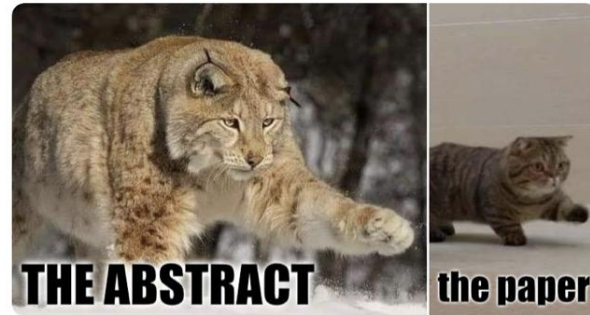
- Multimodal framework overview
- LLMs: Large Language Models
- GPT: Generative Pretrained Transformer
- CLIP: Contrastive Language-Image Pre-training
- LLaVA: Large Language and Vision Assistant
- Gemini: A Family of Highly Capable Multimodal Models

MULTIMODAL FOUNDATION MODELS (MFM)

- An MFM is a large-scale model capable of processing multiple types of inputs, where each *modality* refers to a specific type of data:
 - *Text*
 - *Images*
 - *Audio*
 - *Videos*
 -
- A classic application of MFMs is *Visual Question Answering (VQA)*
 - is a task where, given an **input image** and a **natural language question** about the image, the model generates a **natural language answer**.
- Other applications: Text-Image Retrieval, Image Generation, etc.



MULTIMODAL FOUNDATION MODELS - VQA EXAMPLE



Explain the meme

This meme humorously captures the common experience many researchers and readers encounter when reading academic papers. It shows a powerful, fierce lynx labeled "THE ABSTRACT" on the left and a smaller, cautious-looking domestic cat labeled "the paper" on the right.

The joke here is that an abstract (the brief summary at the beginning of a paper) often makes the research sound exciting, groundbreaking, or impactful, like a "fierce lynx." However, the actual content of the paper may feel less impressive, or overly complex, dense, or even disappointing by comparison, represented by the "scaredy-cat" on the right. This is a playful take on how expectations can differ from reality in academic writing.

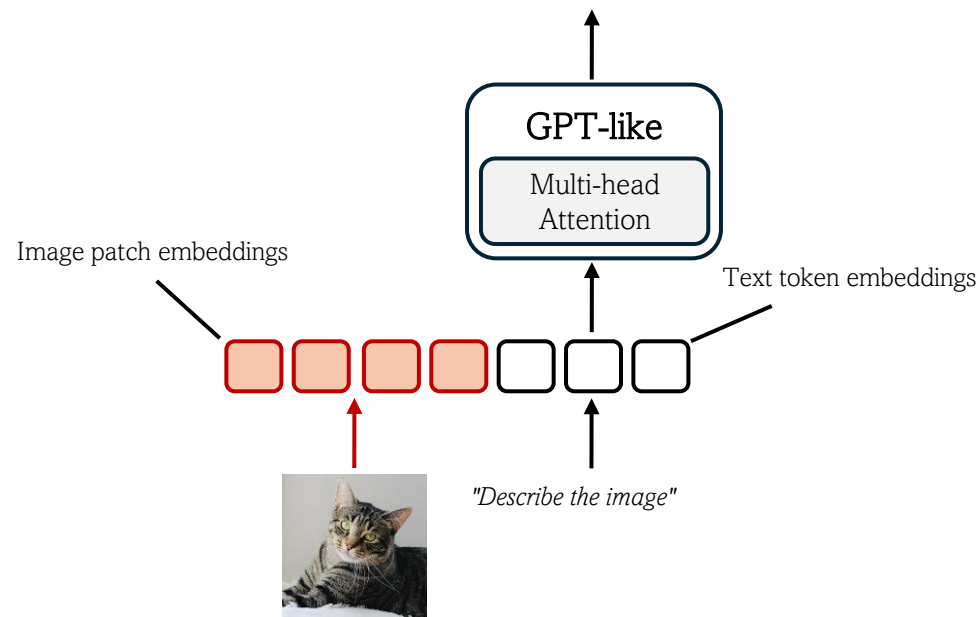
Example use of a Multimodal Foundation Model explaining a meme

MULTIMODAL FRAMEWORK OVERVIEW

COMMON APPROACHES TO BUILDING MFMS

- Focusing on vision-language modalities, there are 2 main approaches to building MFMs:
 - A. Unified Embedding with Decoder architecture
 - B. Cross-Modality Attention architecture

Method A: Unified Embedding Decoder architecture



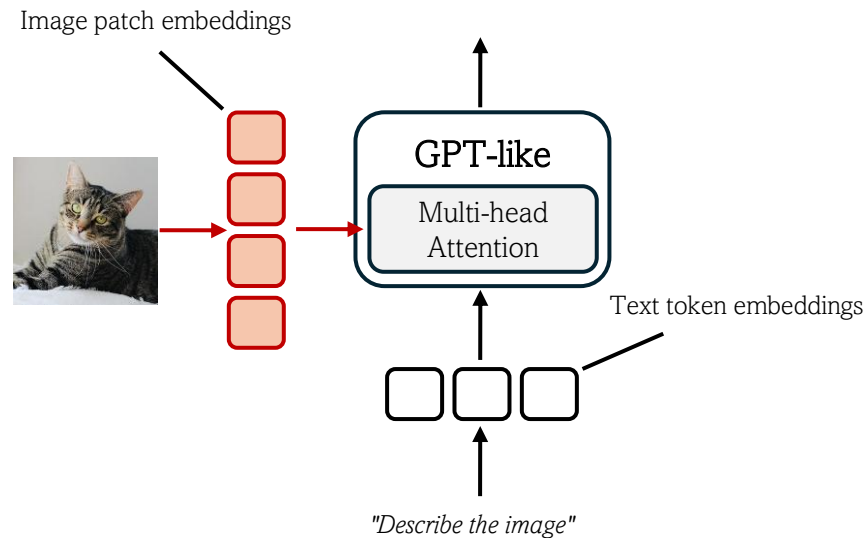
An approach where inputs from different modalities (e.g., image and text) are mapped into a **shared embedding** space, and a **decoder** (usually language-based) **generates** the output, such as an answer or caption, based on this unified representation.

MULTIMODAL FRAMEWORK OVERVIEW

COMMON APPROACHES TO BUILDING MFMS

- Focusing on vision-language modalities, there are 2 main approaches to building MFMs:
 - A. Unified Embedding with Decoder architecture
 - B. Cross-Modality Attention architecture

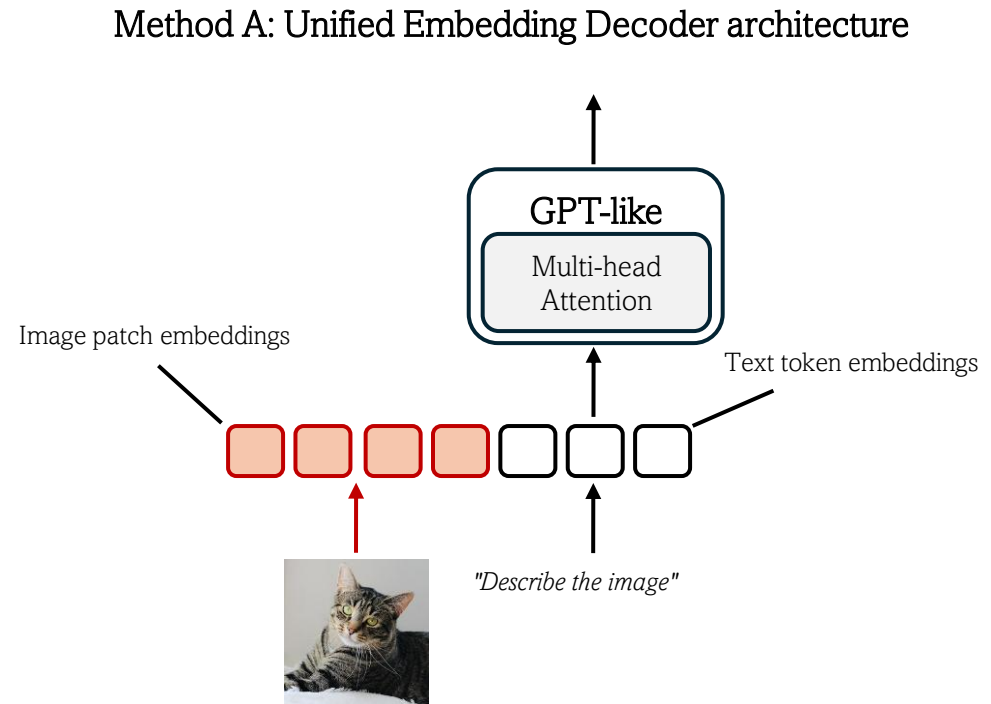
Method B: Cross-Modality Attention architecture



An approach where the model uses **attention mechanisms** to directly exchange and integrate information between modalities (e.g., vision and language), allowing one modality (like text) to attend to features from another (like image) during processing.

UNIFIED EMBEDDING DECODER ARCHITECTURE APPROACH

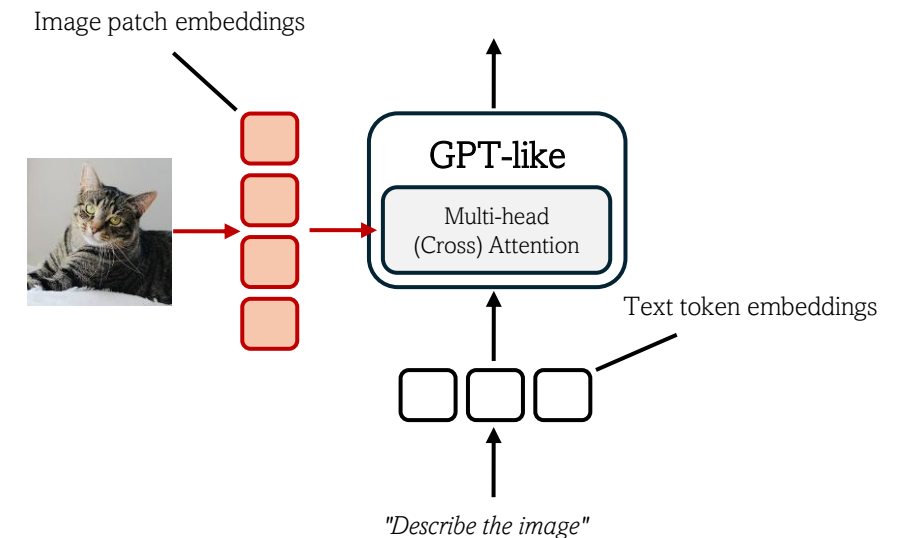
- Uses a **single decoder model**, like an unmodified LLM architecture such as GPT-2.
- **Images are converted into tokens** (i.e., vectors) with the same embedding size (i.e., dimension) as the original text tokens.
- This allows the **LLM to process both text and image input tokens together** after concatenation.
- Ex: LLaVA [1], MiniGPT-4 [2], Qwen-VL [3]



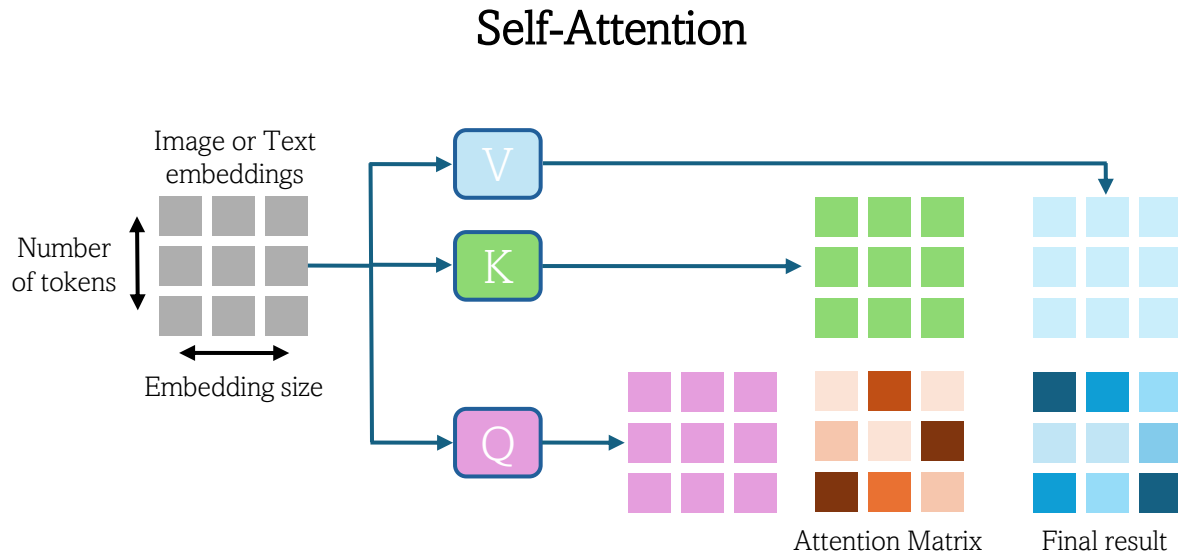
CROSS-MODALITY ATTENTION ARCHITECTURE APPROACH

- The **Cross-Modality Attention** architecture employs a **Cross-Attention** mechanism to **integrate image and text embeddings** directly within the attention layer.
- **Cross-Attention**: a mechanism that allows one set of inputs (e.g., text embeddings) to focus/attend to another set of inputs (e.g., image embeddings), enabling **information flow across different modalities**
- This allows the **LLM** to **attend to visual regions relevant to the text**, to reason about language based on visual content
- Ex: Llama 3.2-V [4], Aria [5], NVLM-X [6], etc.

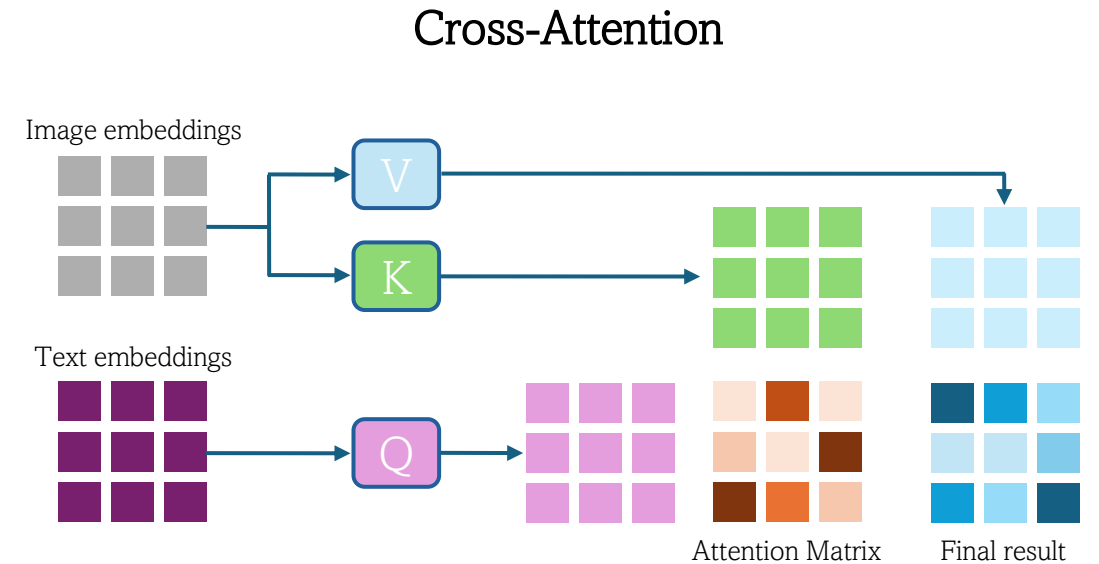
Method B: Cross-Modality Attention architecture



SELF-ATTENTION VERSUS CROSS-ATTENTION



$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$



$$\text{Cross-Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

- While **Self-Attention** captures relationships within a modality (e.g., image-to-image or text-to-text), **Cross-Attention** fuses information across different modalities.

MULTIMODAL FRAMEWORK OVERVIEW

MAIN BUILDING BLOCKS

- Focusing on **vision-language MFMs**, we can identify 3 main building blocks:

- Image Encoder

- Text Tokenizer

- GPT-like LLM

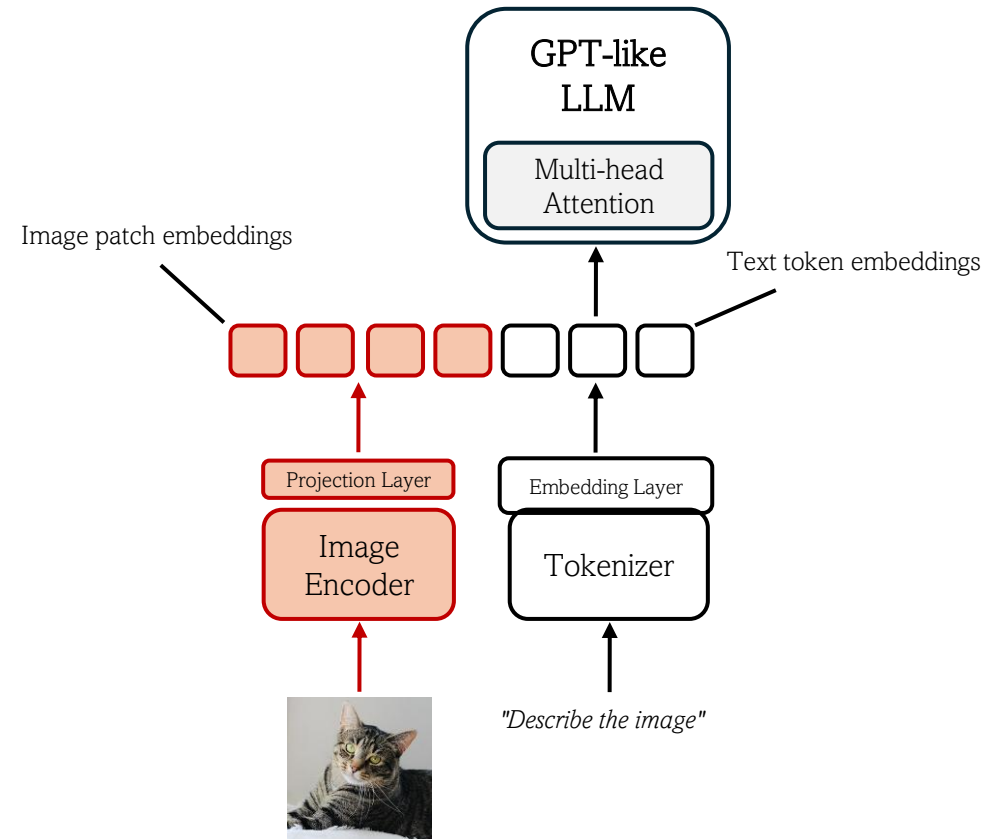


IMAGE ENCODER

- To process an image, we **split** it into smaller patches
 - Patches are flattened and projected into **Patch Embeddings**.
- The **Patch Embeddings** are encoded by a pre-trained Vision Transformer (ViT [7]).
- The final **output** of the Transformer is a set of **Patch Embeddings**
 - They capture both local and global context
- We cannot directly input Patch Embeddings into the LLM, why?

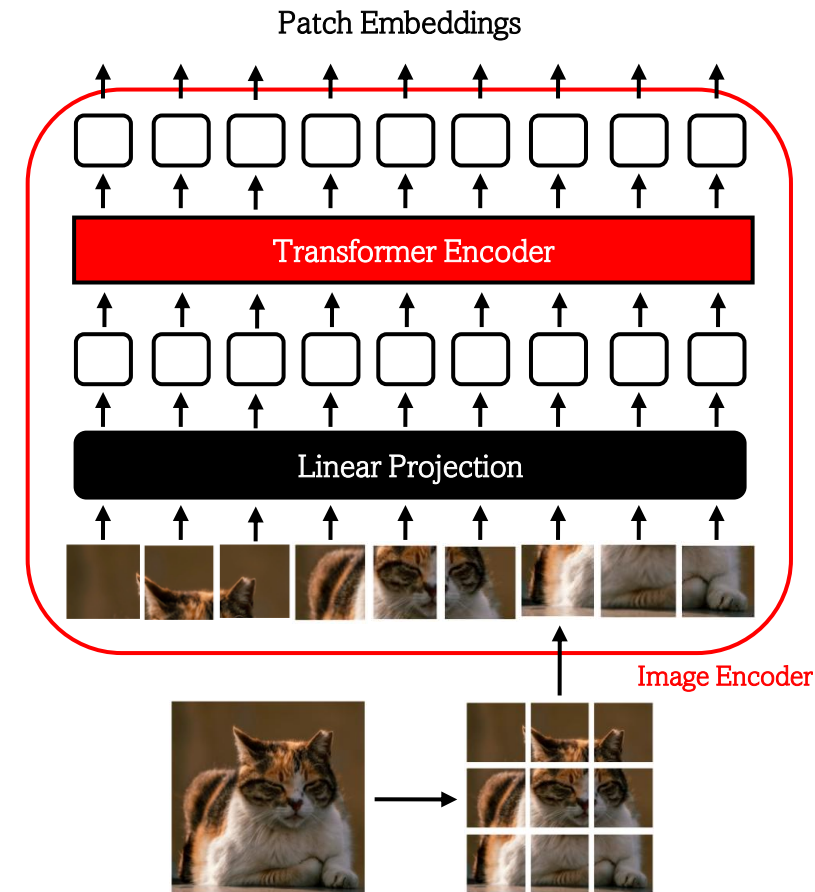
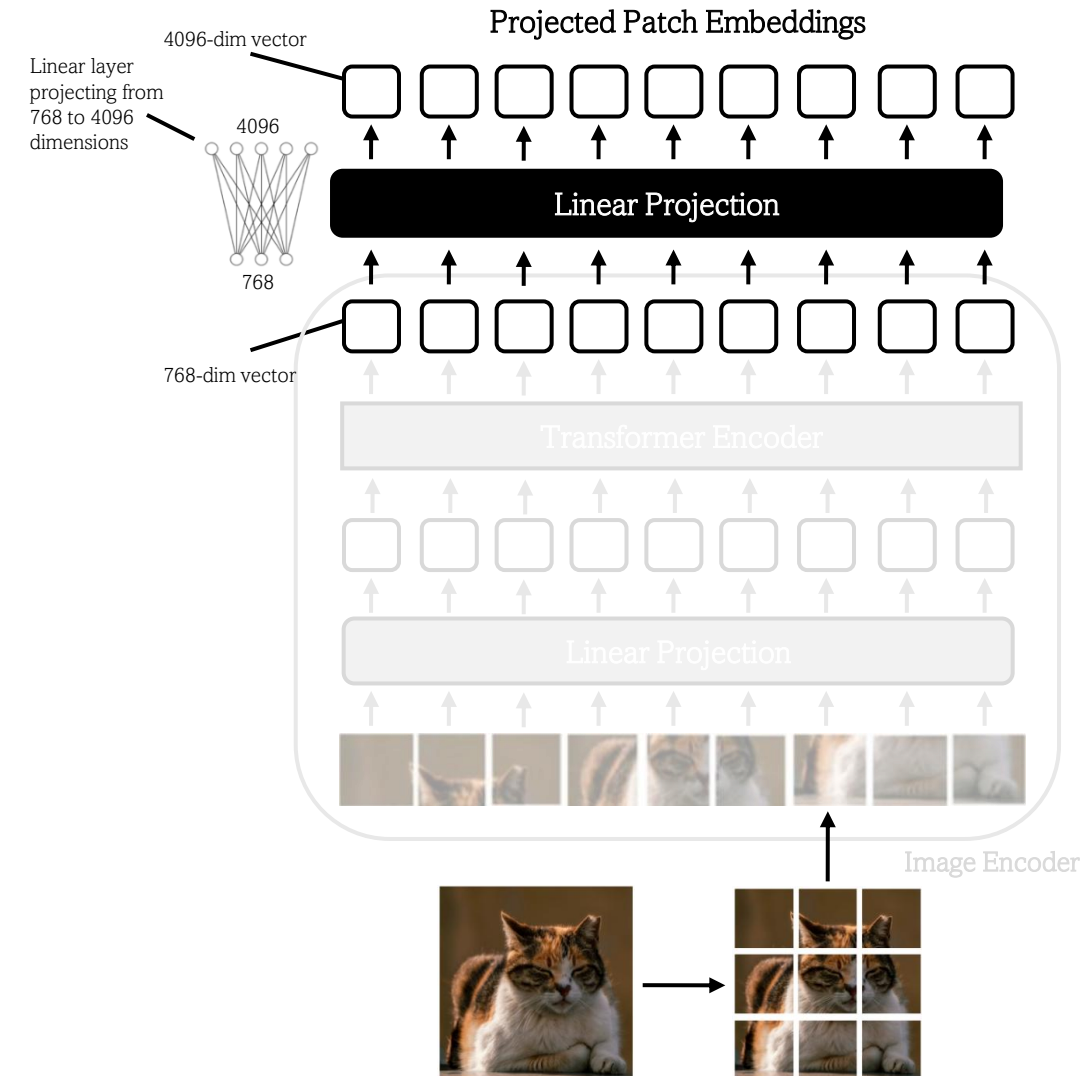


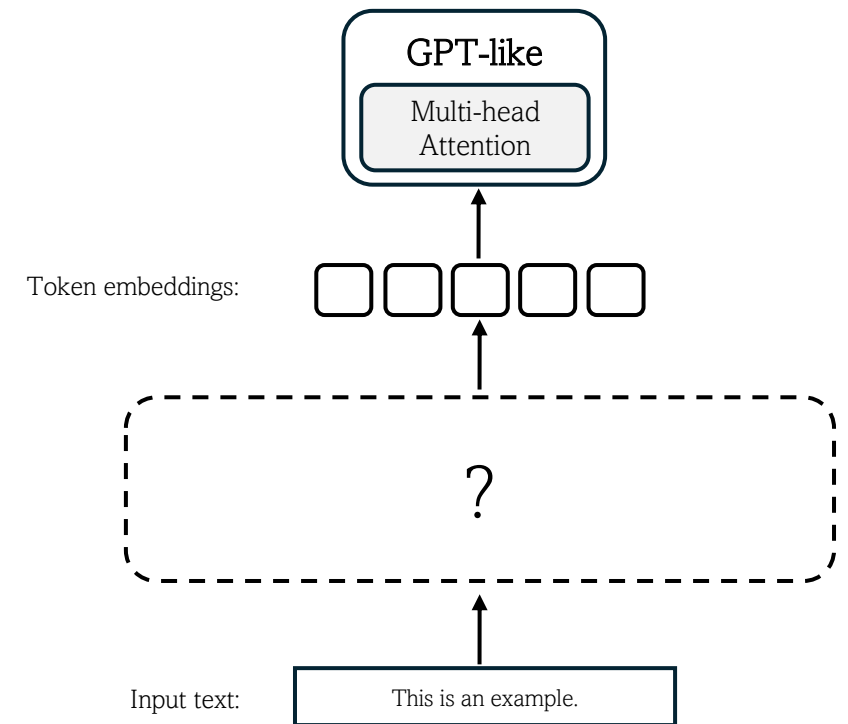
IMAGE ENCODER

- We cannot directly input Patch Embeddings into the LLM:
 - Patch embeddings may have an embedding size that is not compatible with the LLM embedding size.
 - They represent information in a space the LLM is not trained to interpret or understand:
- We must project Patch Embeddings to:
 - Have the same LLM embedding size.
 - Be aligned with the LLM language-based representation.



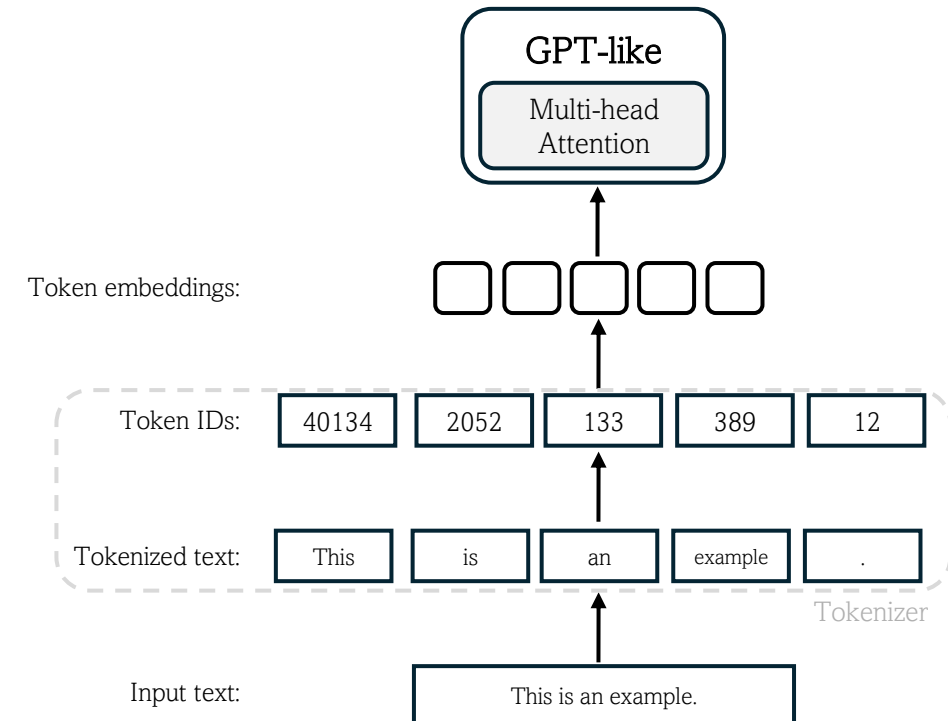
LLM INPUT TEXT FORMAT

- LLMs cannot process raw text directly.
 - Since text is categorical, it is not compatible with the mathematical operations used in neural networks.
- We need a way to **represent words as continuous-valued vectors**.
 - The concept of converting data into a vector format is often referred to as *embedding*.
 - An *embedding* is a mapping from discrete objects, such as words, to points in a continuous vector space.
- We can perform this mapping using, in sequence, a **Tokenizer** and an **Embedding Layer**.



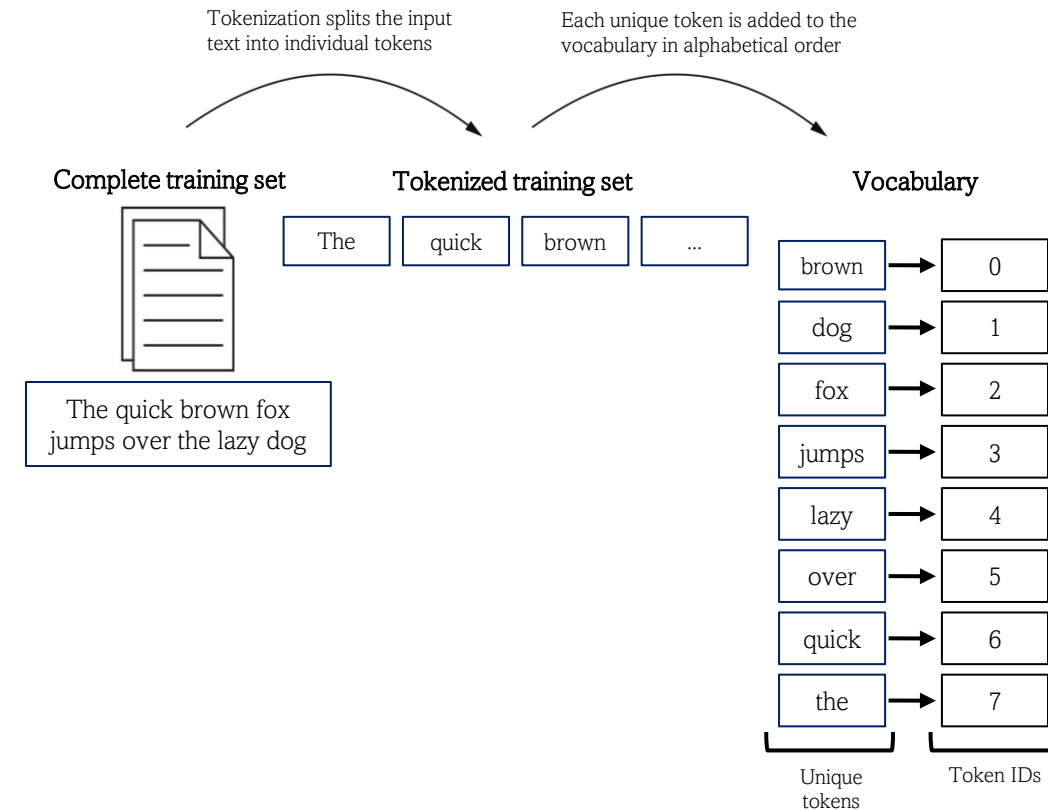
TEXT TOKENIZER

- A **tokenizer splits input text into individual tokens**.
 - There exist many splitting strategies/tokenizers (e.g., Word Tokenization, Byte-Pair Encoding [8])
- Word Tokenizers **tokenize each word separately** by splitting the sentence with spaces.
 - Punctuation characters are also treated as single tokens.
- After tokenization, the tokenizer **converts each string token to an integer** to produce the **token IDs**.
 - To map string tokens into token IDs, we have to build a **Vocabulary** first.



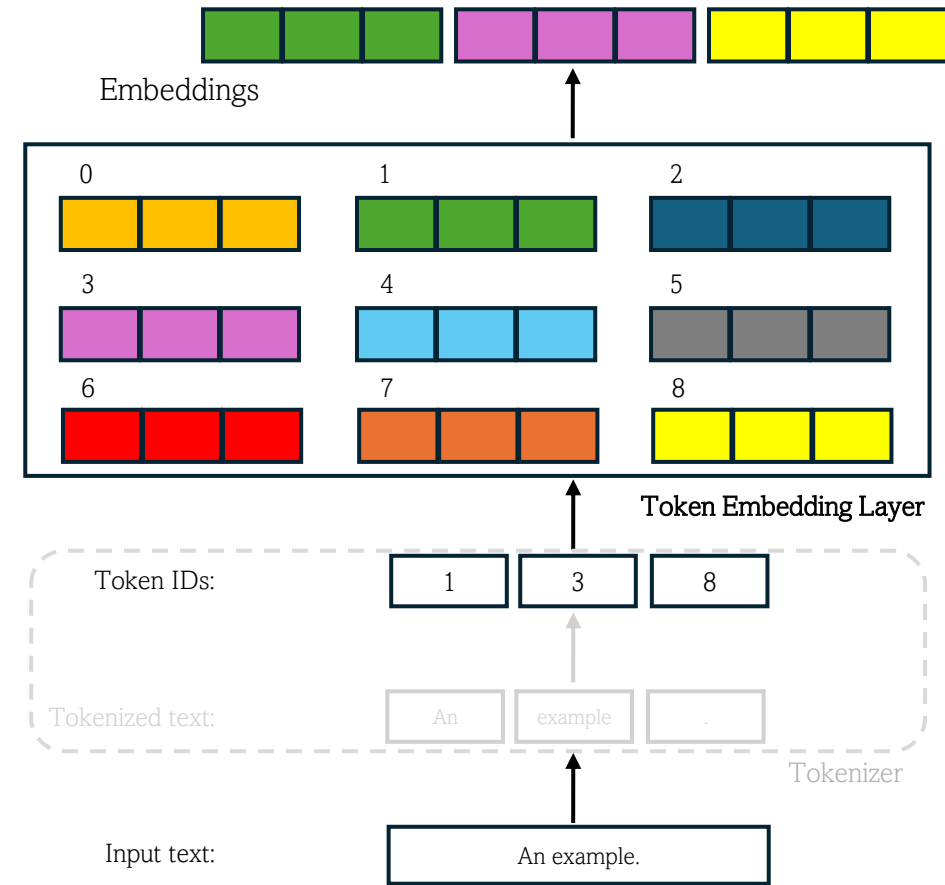
TEXT TOKENIZER

- **Vocabulary**: mapping from string tokens to token IDs
 - Defines how to map each word to a unique integer
- We can build a **Vocabulary** as a **Dictionary**
 - e.g., {"brown": 0, "dog": 1, "fox": 2, ...}
- To **populate the Dictionary**, we **tokenize the entire text** in a training dataset into individual tokens
 - Tokens are sorted alphabetically, and duplicates are removed
- However, this integer representation is not directly usable by the LLM
 - We need to map it into a continuous embedding space, how?



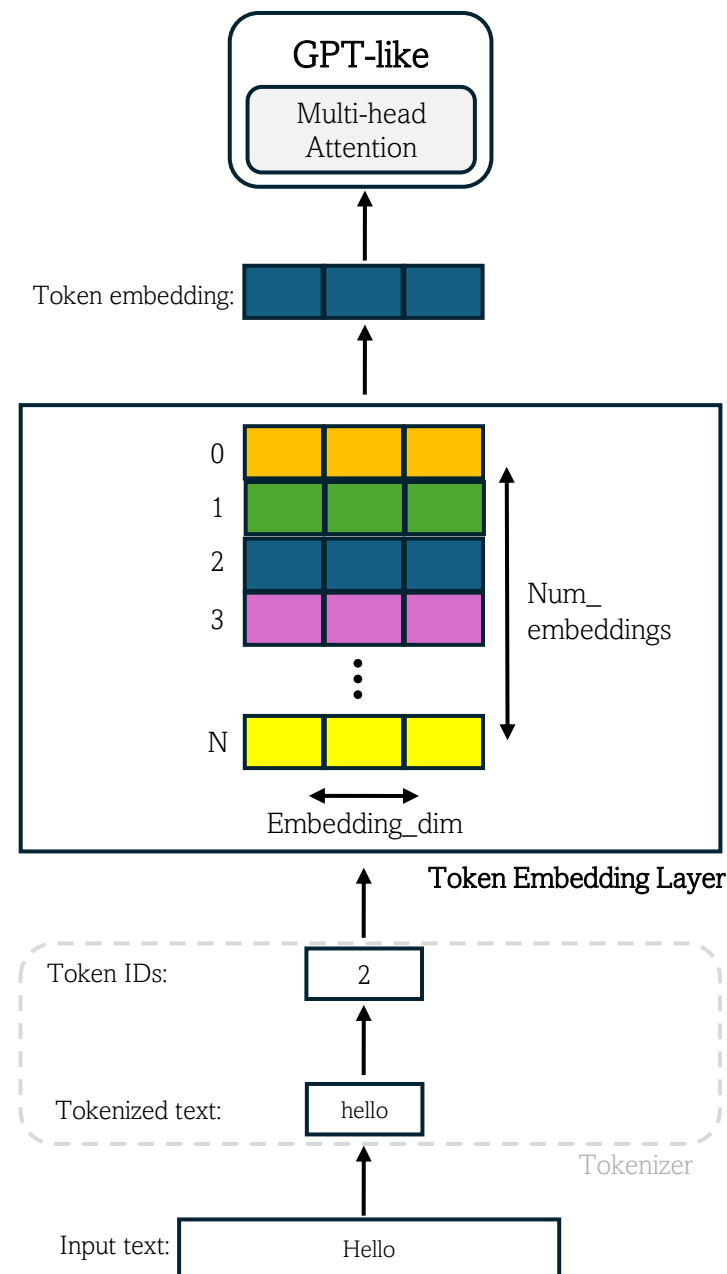
TOKEN EMBEDDING LAYER

- **Token Embedding Layer**: a lookup mechanism that maps integer IDs to high-dimensional vectors
 - When the layer receives an integer index, it returns the corresponding embedding vector from its internal table
 - It maps each word ID to its vector representation
- **Text Embeddings are learnable**
 - Embedding weights are typically initialized randomly and adjusted during training through backpropagation, exactly like other model parameters are trained
- How is a Token Embedding layer implemented?



TOKEN EMBEDDING LAYER

- A **Token Embedding Layer** can be implemented as a **matrix** of size $(num_embeddings, embedding_dim)$
 - $num_embeddings$ is the number of unique items (words)
 - $embedding_dim$ is the dimensionality of each embedding vector
- When the Token Embedding Layer is provided with an index (i.e., a Token ID), it **accesses the row in the matrix at the position specified by the index**
- *Now that we have covered how to prepare inputs for an LLM, let's look into the LLM!*





LLMs: LARGE LANGUAGE MODELS

LLMs: LARGE LANGUAGE MODELS

INTRODUCTION

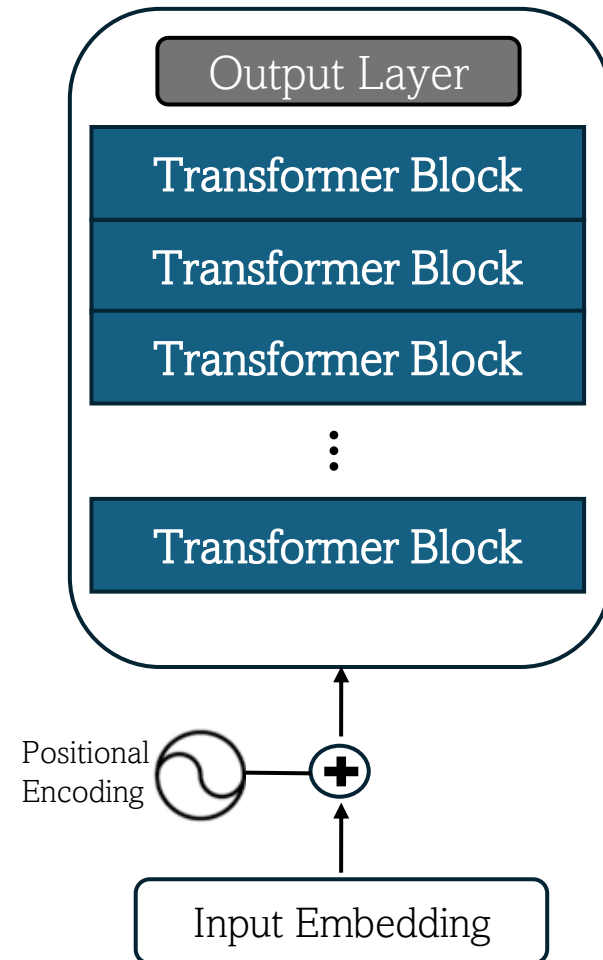
- **Large Language Model (LLM)**: a type of **deep learning model** designed to understand, generate, and work with human language
 - Capable of performing a variety of tasks (Code generation, Chatbots, etc.)
 - Based on the Transformer architecture
 - Trained on vast datasets of text
 - Initially designed to work with text-only
- What's inside an LLM?



LLMs: LARGE LANGUAGE MODELS

ARCHITECTURE OVERVIEW

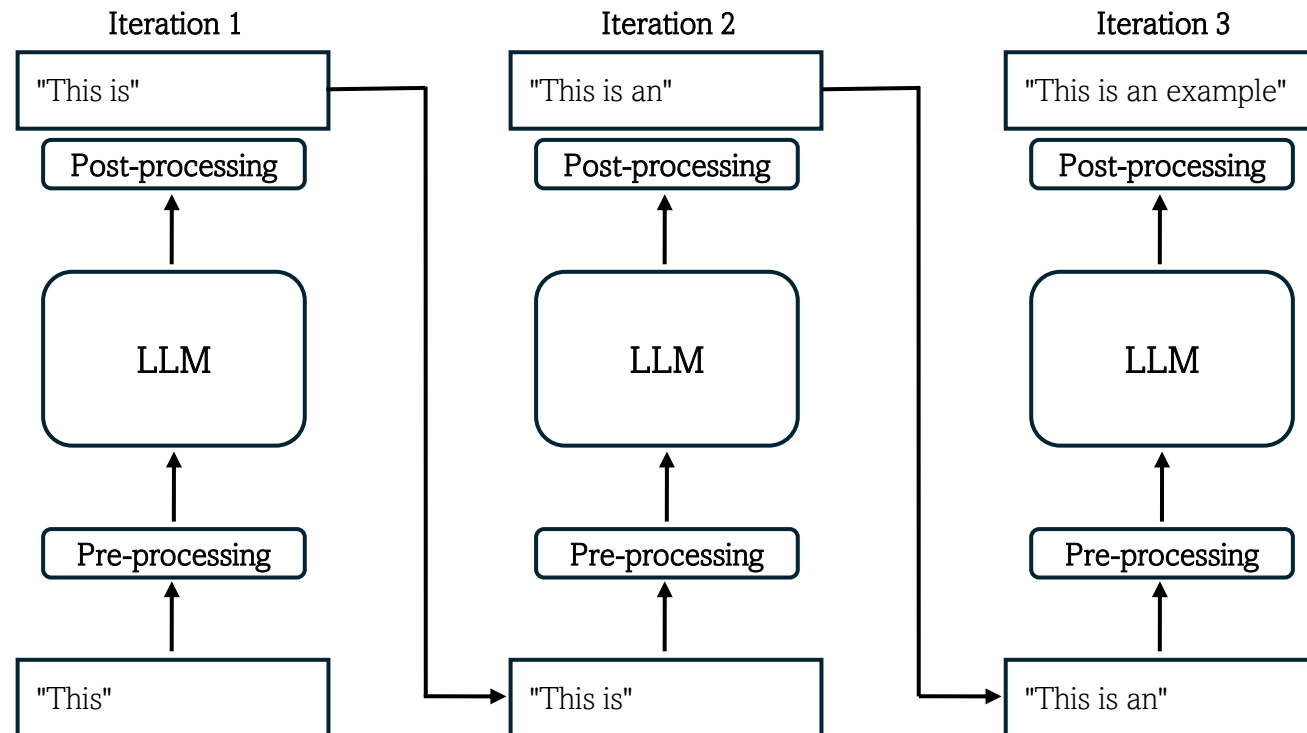
- LLMs are composed of multiple Transformer blocks.
- Each LLM block leverages **Masked Self-Attention (MSA)**:
 - Which is a variant of Self-Attention
 - Similarly to Self-Attention, MSA is used to **model relationships between different tokens in the input sequence**.
- How does an LLM work / perform inference?



LLMs: LARGE LANGUAGE MODELS

AUTOREGRESSIVE MODELING

- LLM models **are autoregressive models**:
 - Incorporate their previous outputs as inputs for the future.
 - Each new word is chosen based on the sequence that precedes it.



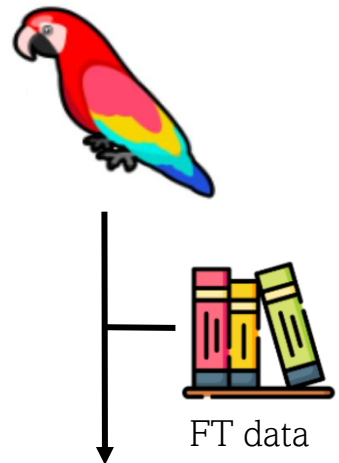
LLMs: LARGE LANGUAGE MODELS

TRAINING PHASES

LLMs are trained in 2 steps:

- **LLM Pretraining:** This step **creates an initial pre-trained LLM**, often called a base or foundation model
 - This step provides the LLM with a "broad" understanding of language.
 - After this step, the model is only capable of text completion, which means, just finishing a half-written sentence provided by a user.
- **Instruction Fine-Tuning:** This step fine-tunes the pre-trained base LLM to increase accuracy in specific applications.
 - After this step, the model will not only complete text but also understand it and generate responses based on *specific instructions (e.g., summarization, translation, question answering, etc.)*.

Base LLM



Fine-tuned LLM

LLMs: LARGE LANGUAGE MODELS

TRAINING TASK

- LLMs are **trained on** a simple **Next-Token Prediction (NTP)** task
 - The model learns to predict the upcoming word in a sentence by looking at the words that have come before it.
 - This helps the model understand how words and phrases typically fit together in language, forming a "foundational" understanding of language during pre-training.

**The model is simply trained to
predict the next word**

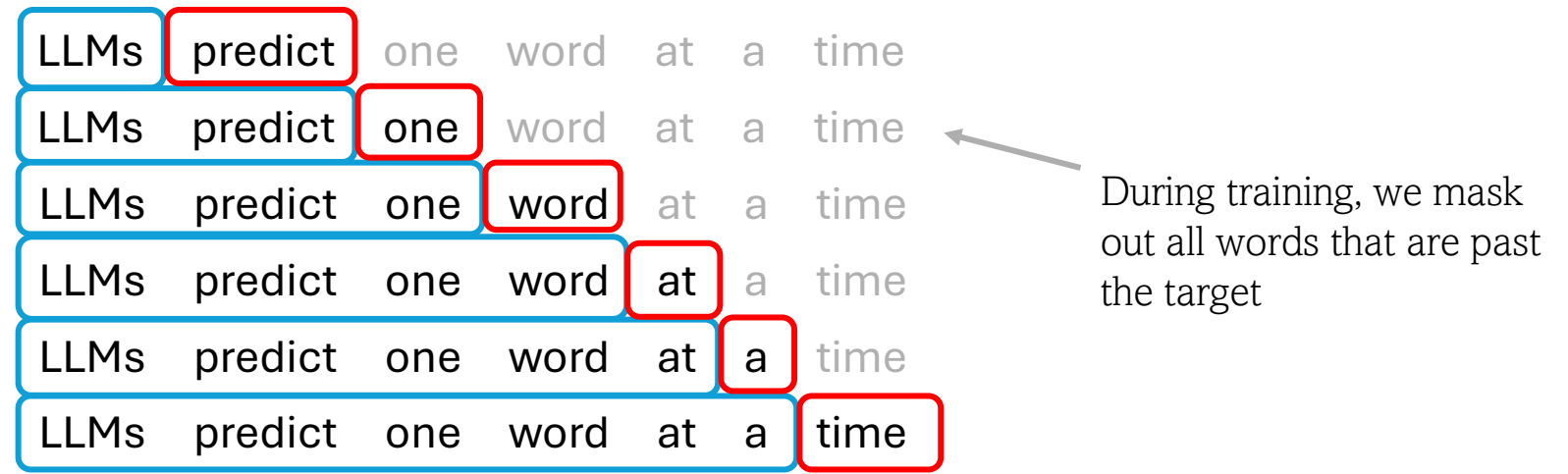


- The **Next-Token Prediction** task is a form of **self-supervised learning**.
 - We use the structure of the data itself instead of collecting labels.
 - We use the next word in a sentence as the label that the LLM has to predict.
 - The labels (next token/word) are already part of the text — no human annotation is needed.
- How can an LLM output a whole sentence just by next-token prediction?

LLMs: LARGE LANGUAGE MODELS

TRAINING DATA FORMAT

- Input-target pairs required to train an LLM are built with a sliding window approach
 - We create **input** samples by keeping the window enlarged to the right
 - For each input sample, the **target** is just the input shifted by one position



Note that the text shown in this figure must undergo tokenization before the LLM can process it.

LLMs: LARGE LANGUAGE MODELS

MASKED SELF-ATTENTION (MSA)

- In **standard Self-Attention**, each token in the input sequence is allowed to **attend to all other tokens**.
 - This is beneficial in tasks that require processing of the entire input, e.g., *sentence translation* or *sentiment analysis*.
- However, this capability **can lead to information leakage** in language model training for text generation.
 - Information leakage occurs when a model, during training, has access to future tokens it will need to predict.
- We do not want this capability, as the goal of **Next-Token Prediction** is to mimic **sequential prediction**, where **each word is predicted based only on previous words**.

MASKED SELF-ATTENTION (MSA)

Self-Attention
Binary Mask (no mask)

	The	quick	brown	fox
The	1	1	1	1
quick	1	1	1	1
brown	1	1	1	1
fox	1	1	1	1

Standard Self-Attention: every token can attend to every other token in the sequence

Masked Self-Attention
Binary Mask (lower triangular)

	The	quick	brown	fox
The	1	0	0	0
quick	1	1	0	0
brown	1	1	1	0
fox	1	1	1	1

Masked Self-Attention: each token attends only to itself and previously seen tokens (0 is semantically zero here, but not in practice)

- We use **Masked Self-Attention** in LLMs because it makes sure each word is predicted from the past words only, aligning with *Next-Token Prediction*.

MASKED SELF-ATTENTION (MSA)

- How to implement Masked Self-Attention? Recall that $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$

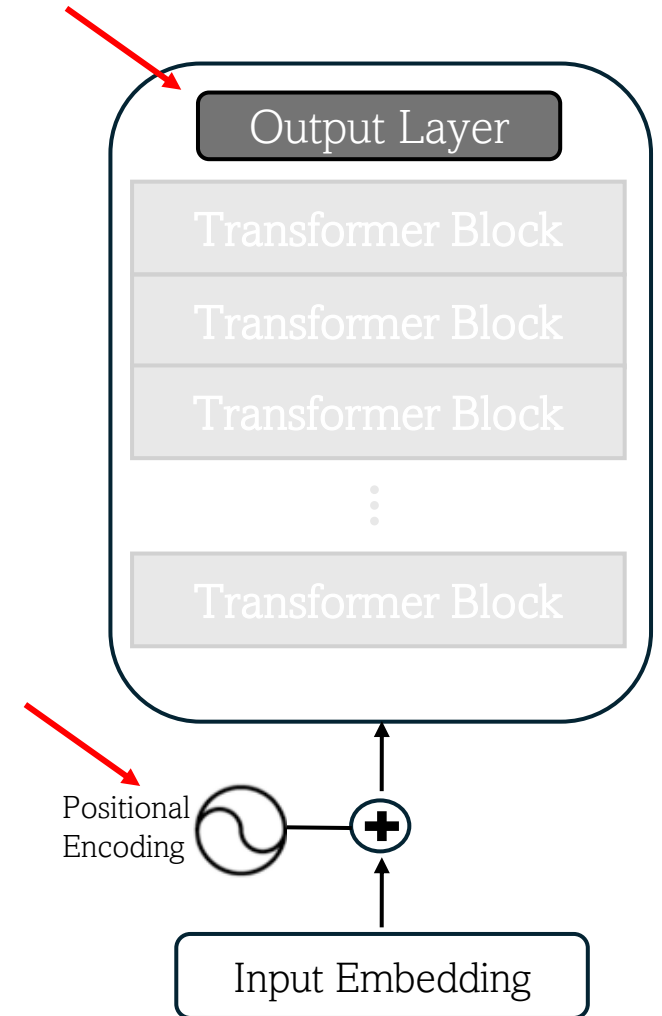
A (Attention matrix)						A_masked					Softmax(A_masked)					
	The	quick	brown	fox		The	quick	brown	fox		The	quick	brown	fox		
The	0.5	0.7	-0.6	1.1	→ mask	The	0.5	-∞	-∞	-∞	→ Softmax	The	1.0000	0.0000	0.0000	0.0000
quick	1.2	0.3	1.0	-1.3		quick	1.2	0.3	-∞	-∞		quick	0.7110	0.2890	0.0000	0.0000
brown	-0.4	2.1	0.9	0.8		brown	-0.4	2.1	0.9	-∞		brown	0.0593	0.7229	0.2177	0.0000
fox	1.0	-1.5	0.2	2.4		fox	1.0	-1.5	0.2	2.4		fox	0.1790	0.0147	0.0804	0.7259

- We **cannot use 0 for masking** as it still contributes to the Softmax calculation ($e^0=1$ contributes to the sum in Softmax denominator)
- Instead, we use **minus infinity as a masking value** as it reduces the Softmax output of all masked tokens to zero ($e^{-\infty}=0$), ensuring they are ignored.

LLMs: LARGE LANGUAGE MODELS

POSITIONAL ENCODING & OUTPUT LAYER

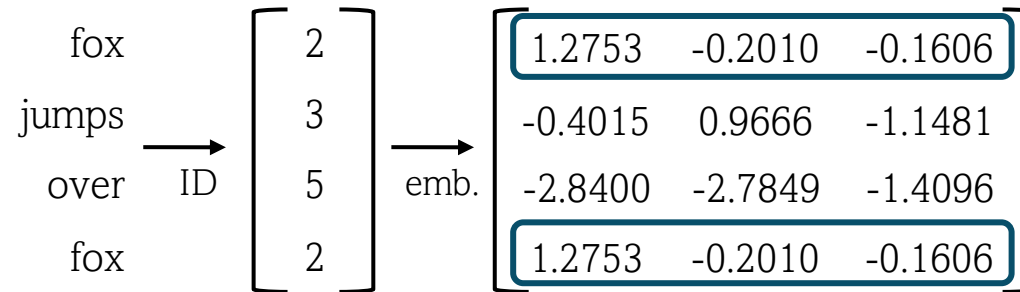
- Back to the architecture, there are 2 components we have yet to analyze:
 - Positional Encoding
 - LLM output layer



LLMs: LARGE LANGUAGE MODELS

POSITIONAL ENCODING

- LLMs' **Self-Attention** mechanism **does not have a notion of position** for tokens.
 - Self-Attention measures relationships between pairs of tokens without considering their positions, focusing purely on token similarity, not on sequence order.

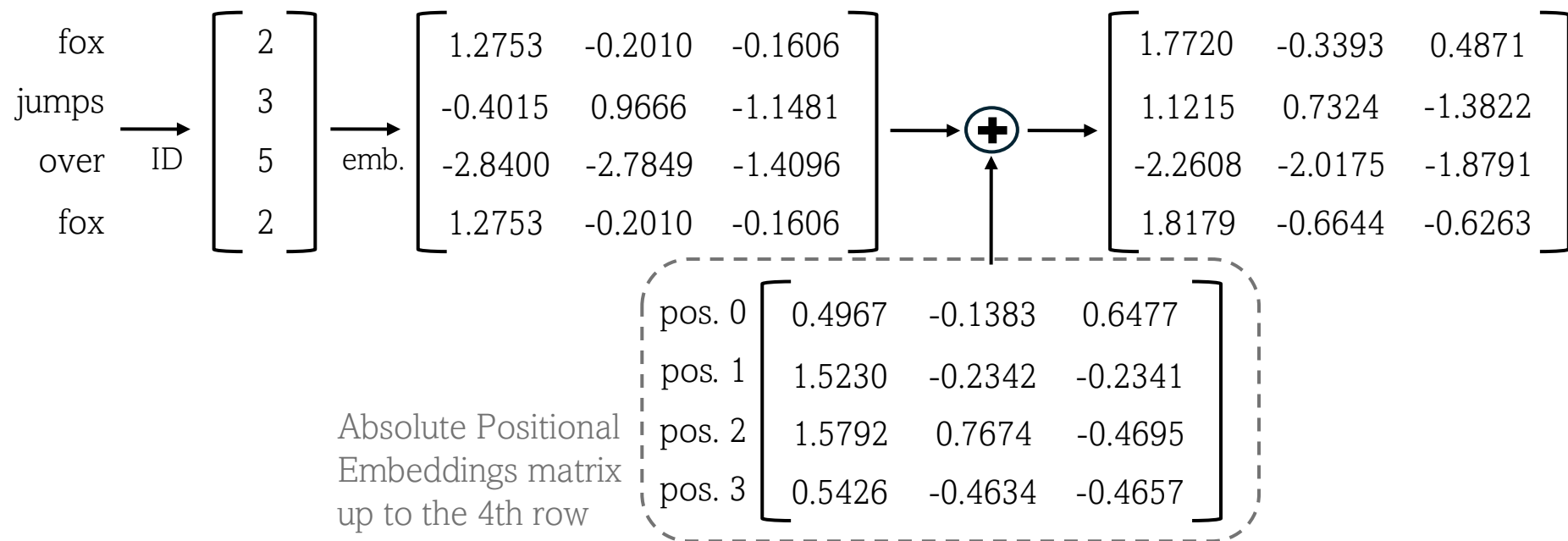


- **Without additional positional awareness**, the **same word** in different contexts would get the **same vector representation** after tokenization.
 - This homogeneity can lead the model to misunderstand, as the meaning of a word can change based on surrounding words and its position in the text.
 - It would be helpful to inject position information into the LLM.

LLMs: LARGE LANGUAGE MODELS

POSITIONAL ENCODING

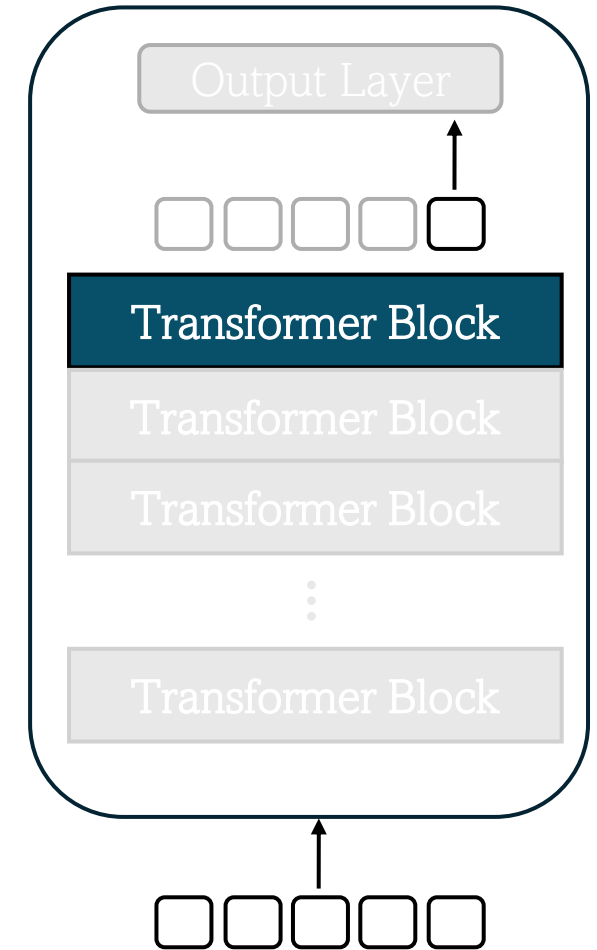
- **Absolute Positional Embeddings:** **Learnable matrix** that **adds** (i.e., not concatenation) **positional context** to the token embeddings before they are processed by LLM's Transformer blocks.
 - The size of the matrix is $(max_sequence_len, embedding_dim)$, where $max_sequence_len$ (e.g., 1024, 2048) is the maximum number of tokens that the LLM can accept as input.



LLMs: LARGE LANGUAGE MODELS

OUTPUT LAYER

- The **last LLM Transformer block** outputs embeddings for all tokens, but **only the embedding of the last token is retained for predicting the next token**.
 - Only the last token embedding is retained since it alone has integrated all preceding context, due to the masked attention mechanism.
 - All the other tokens are discarded.
- After this step, we end up with a single embedding.
- *We need to turn it into an integer ID such that we can map it to the next word, how?*



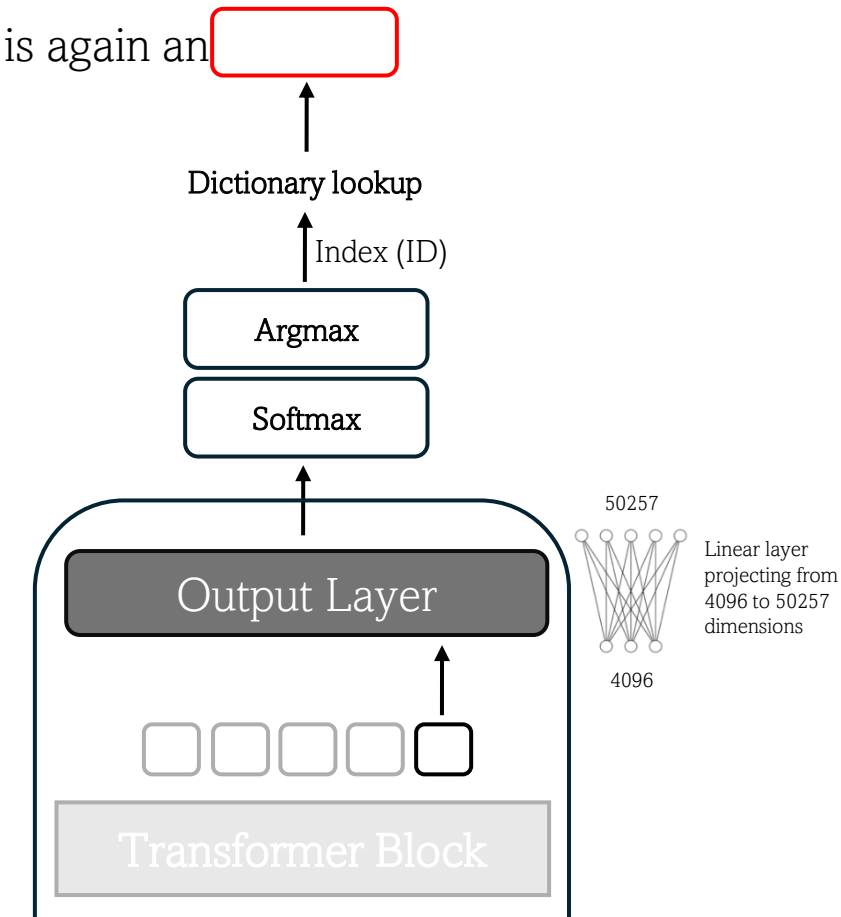
LLMs: LARGE LANGUAGE MODELS

OUTPUT LAYER

- The **last token embedding** is passed through the **Output Layer**, which **projects** it to a **dimension** equal to the **Vocabulary size**.
- The projected embeddings are fed into a **Softmax layer**, which **converts them into a probability distribution** over all possible elements (e.g., words) in the Vocabulary.
- The **index (ID)** of the highest probability is taken, and used to look up the next word in the **Vocabulary**.

This one is again an

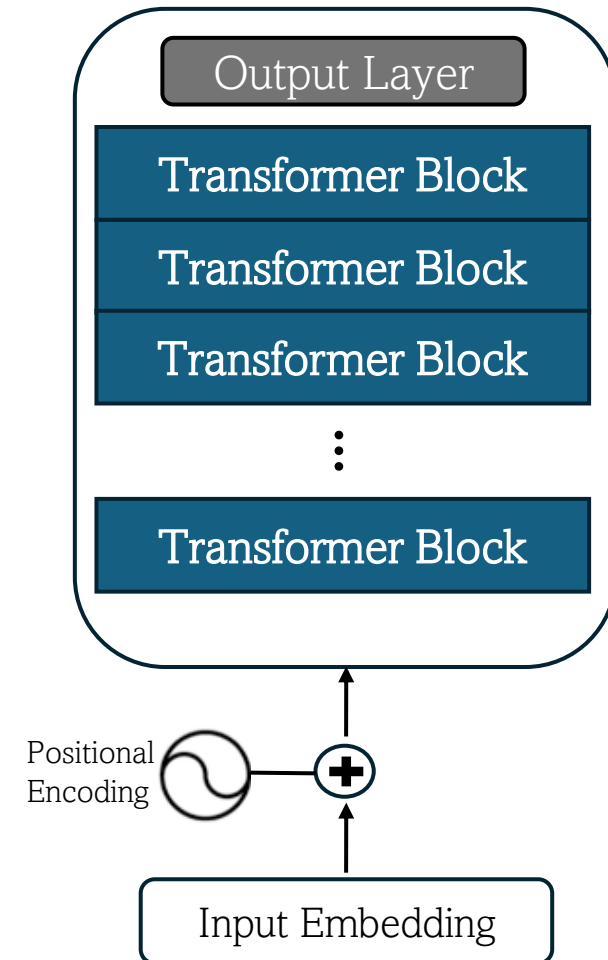
example



LLMs: LARGE LANGUAGE MODELS

CONCLUSIONS - LLM ARCHITECTURE

- We analyzed every *base LLM component*
 - Each component has a set of **learnable weights θ**
- Different LLMs have different architectural setups
 - Different Transformer Blocks sub-components
 - Different number of Blocks
 - Different embedding size
- **Last thing is: what is an LLM training objective?**
 - **How are its weights θ optimized?**



LLMs: LARGE LANGUAGE MODELS

CONCLUSIONS – TRAINING OBJECTIVE

- An LLM training objective is to ensure that the model assigns the highest probability to the correct next token in a sequence, predicting it based on the preceding context.

$$\mathcal{L}(\mathcal{U}) = \sum_{i=1}^N \log \left(P(u_i \mid u_{i-k}, \dots, u_{i-1}, \Theta) \right)$$

Training corpus

Conditional probability of i^{th} token given k preceding tokens and model parameters θ

- This can be implemented by maximizing the overall log probabilities for correct token predictions, given the preceding tokens.

LLMs: LARGE LANGUAGE MODELS

CONCLUSIONS – TRAINING OBJECTIVE

- However, in the context of optimization algorithms used in ML, like gradient descent, the conventional task is to minimize a function.

$$\mathcal{L}(\mathcal{U}) = \underbrace{- \sum_{i=1}^N \log (P(u_i \mid u_{i-k}, \dots, u_{i-1}, \Theta))}_{\text{Conditional probability of } i^{\text{th}} \text{ token given } k \text{ preceding tokens and model parameters } \theta}$$

!!

- By considering the **negative log likelihood**, we turn a maximization problem into a minimization one.
 - Minimizing the negative log likelihood is mathematically equivalent to maximizing the log likelihood.



GPT: GENERATIVE PRETRAINED TRANSFORMER

GPT: GENERATIVE PRETRAINED TRANSFORMER

HISTORY OF GPT MODELS

- GPT is a type of LLM introduced by OpenAI back in 2018:

- GPT-1 [9]: Introduced the Transformer architecture into a **Language Model**. Could be fine-tuned to solve simple tasks (e.g., text classification)
- GPT-2 [10]: marked a **significant leap in terms of model size**. Trained on a dataset of 8 million web pages, WebText [10]. Showed a much better understanding of language
- GPT-3 [11]: **expanded dramatically in model size**. Trained on an even larger corpus that included books and websites. It demonstrated remarkable abilities, such as writing creative fiction, answering questions, and translating languages

GPT-1

2018
117M params

GPT-2

2019
1.5B params

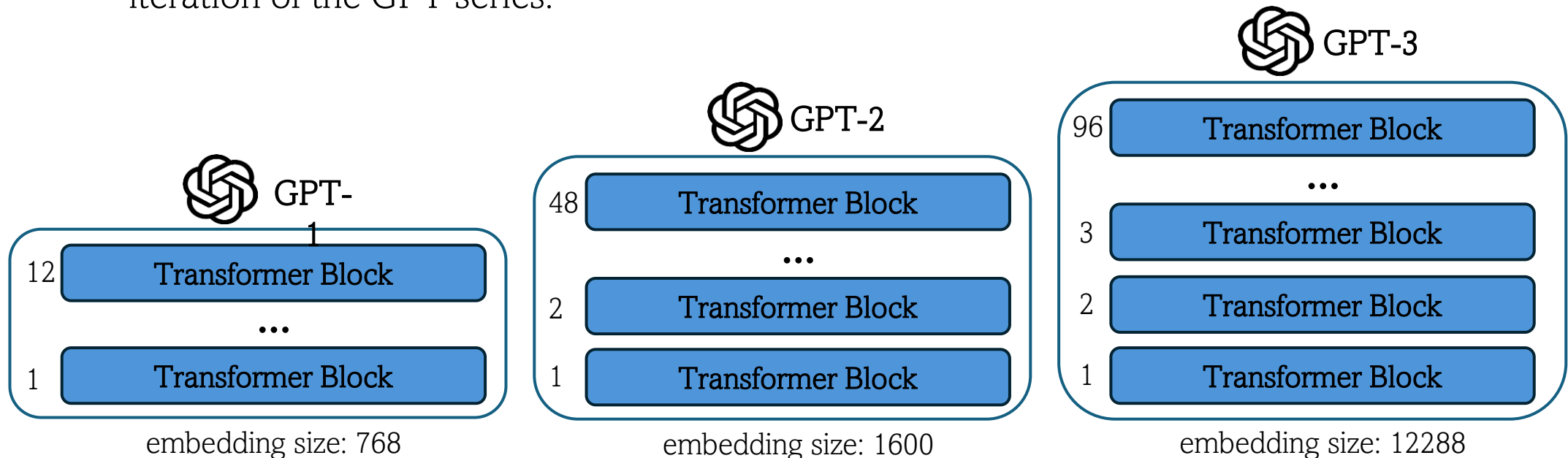
GPT-3

2020
175B params

GPT: GENERATIVE PRETRAINED TRANSFORMER

MODEL SCALING

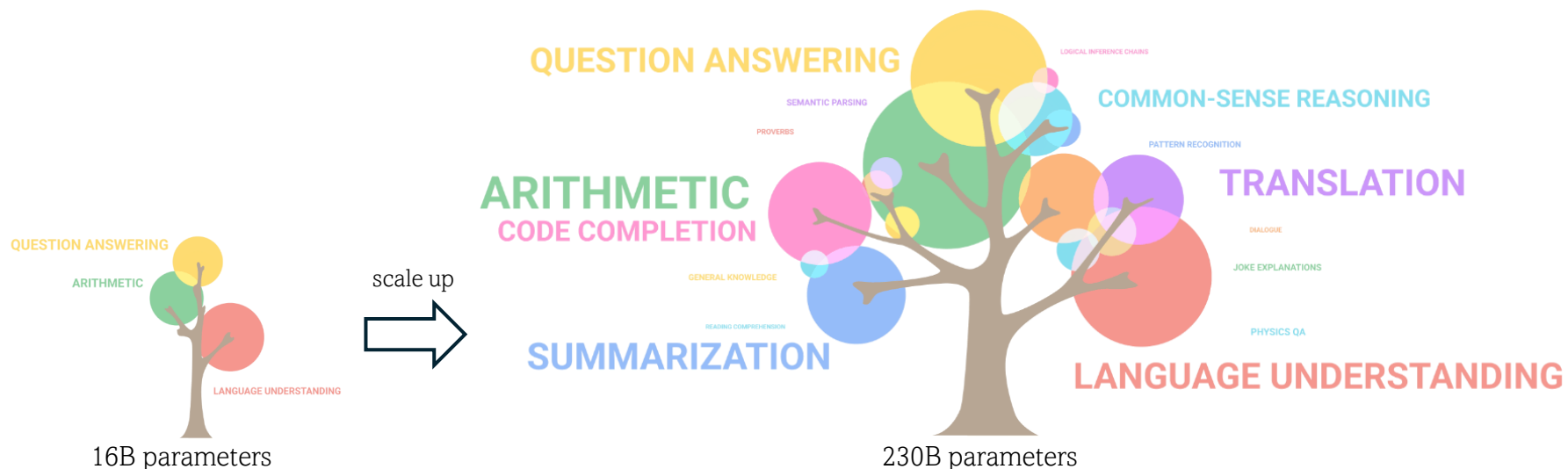
- All GPT models up to GPT-3 share the same fundamental architecture based on the Transformer model.
 - The **primary difference** among GPT-1, GPT-2, and GPT-3 lies in the scale of their implementation, i.e., their **number of parameters**.
 - The number of Transformer blocks and the size of the Embeddings are scaled up with each iteration of the GPT series.



GPT: GENERATIVE PRETRAINED TRANSFORMER

EMERGENT BEHAVIORS

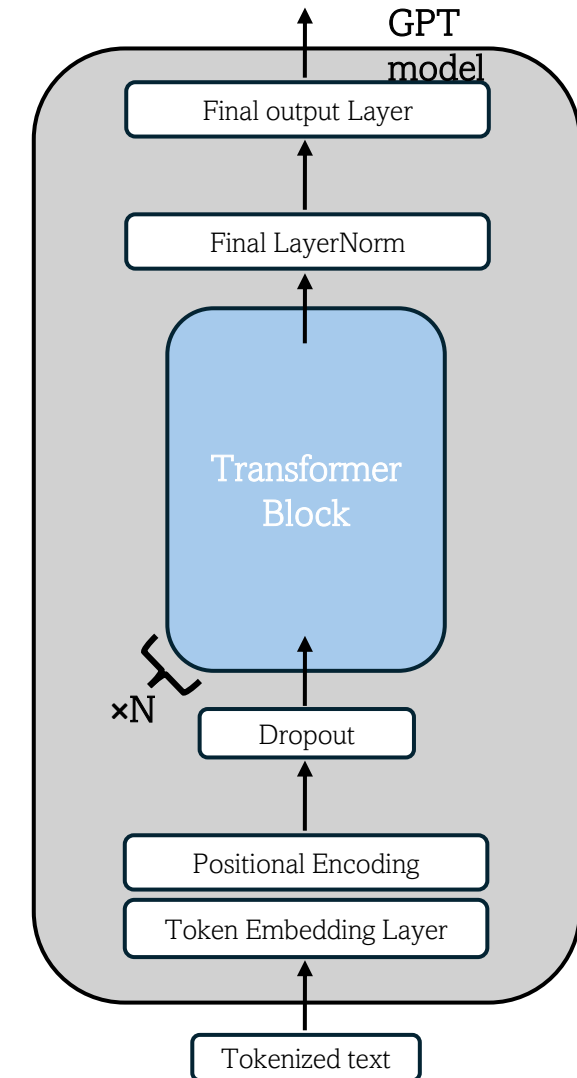
- **GPT models**, despite being aimed at simple next-word prediction, get able to **perform more tasks as they scale up** in model size
 - The ability to perform tasks that the model was not explicitly trained to perform is called an *emergent behavior*, and emerges as a natural consequence of increased model capacity and model exposure to vast quantities of data in diverse contexts.
 - Ex: Given as input: *"English: The dog is playing in the yard. Italian:"* a big enough GPT model would correctly output: *"Il cane sta giocando in giardino"*.



GPT: GENERATIVE PRETRAINED TRANSFORMER

MODEL ARCHITECTURE

- A GPT model contains all LLM components we have seen so far:
 - Embedding Layer
 - Positional Encoding
 - Transformer Blocks
 - Output layer
- The only **differences** are:
 - Initial Dropout layer
 - Final LayerNorm layer
- What's inside a GPT Transformer block?

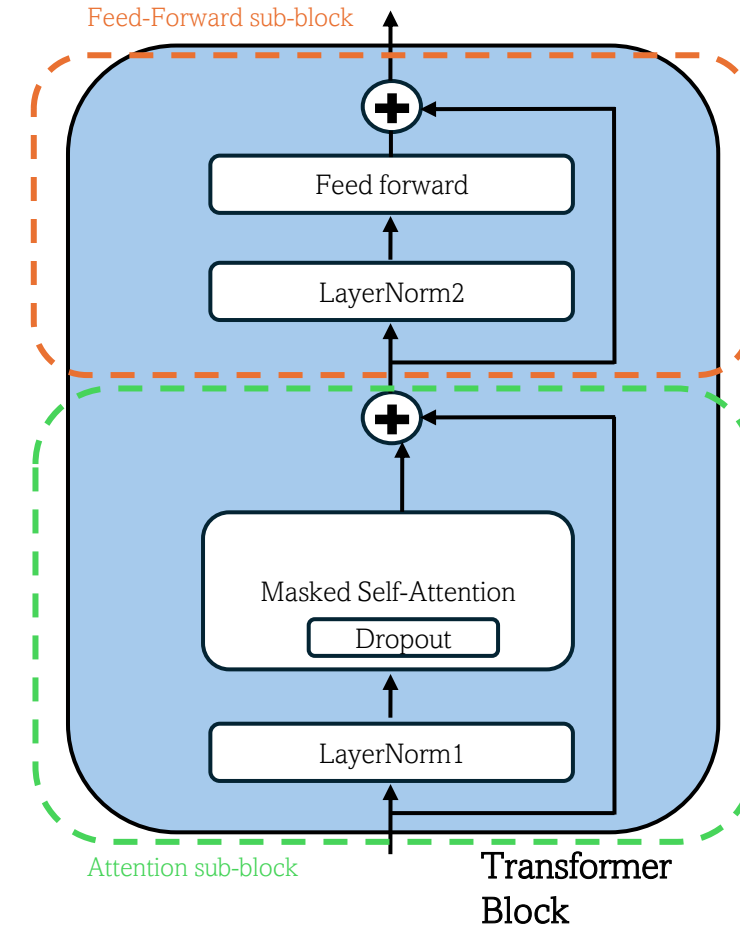


GPT: GENERATIVE PRETRAINED TRANSFORMER

TRANSFORMER BLOCK

Each Transformer Block is split into 2 main parts:

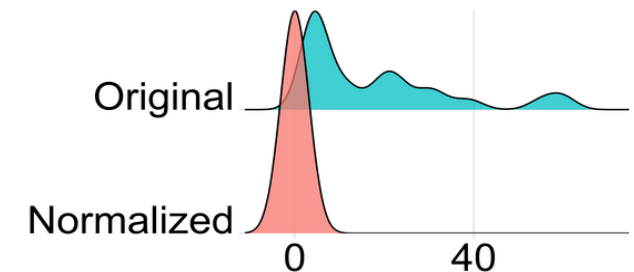
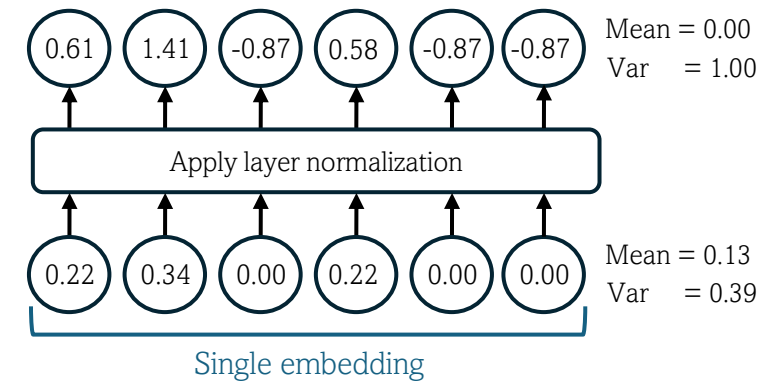
- **Attention Sub-block**
 - Enables the model to **blend information** from earlier tokens, capturing dependencies such as syntax.
 - Uses a **residual connection** (adds the input of the attention layer to its output) to preserve the original tokens' identity (after Attention mixing).
- **Feed-Forward Sub-block**
 - **Re-encode separately each token** (no token mixing, no cross-token communication) representation to add depth and complexity, without mixing them.
 - Uses a **residual connection** (input is added to the output of the MLP) to ensure that the original signal from the attention block is preserved, while allowing for further transformation.



GPT: GENERATIVE PRETRAINED TRANSFORMER

LAYER NORMALIZATION

- Training deep networks can lead to instabilities or problems like **vanishing gradients**.
- **Layer normalization** adjusts the **activations** (or outputs) of a layer to have **zero mean and unit variance**.
 - Without normalization, activations could grow to very large or small values, causing gradients to explode or vanish.
 - With normalization, all **activations are kept within a more controlled range**, leading to more balanced gradients and **more stable training**.
 - Normalization is **computed across the embedding dimension**: each token is normalized (i.e., rebalanced) independently.

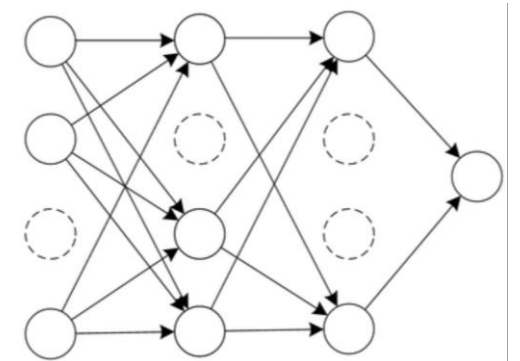


After normalization, the original wide and skewed distribution gets tighter and more centered around zero

GPT: GENERATIVE PRETRAINED TRANSFORMER

DROPOUT

- Recall that **Dropout** [12] is a **regularization technique** where **randomly selected units** (e.g., neurons) **are ignored** during training, effectively "dropping" them out.
 - This method helps prevent **overfitting** by ensuring that a model does not become overly reliant on any specific set of units.
 - The set of ignored units is randomly selected at each training iteration.
- In **GPT** models, **Dropout** is applied in the **Attention Mechanism** after calculating the **Attention weights** (i.e., softmax).
 - Recall that Dropout is only used during training and is disabled afterwards.

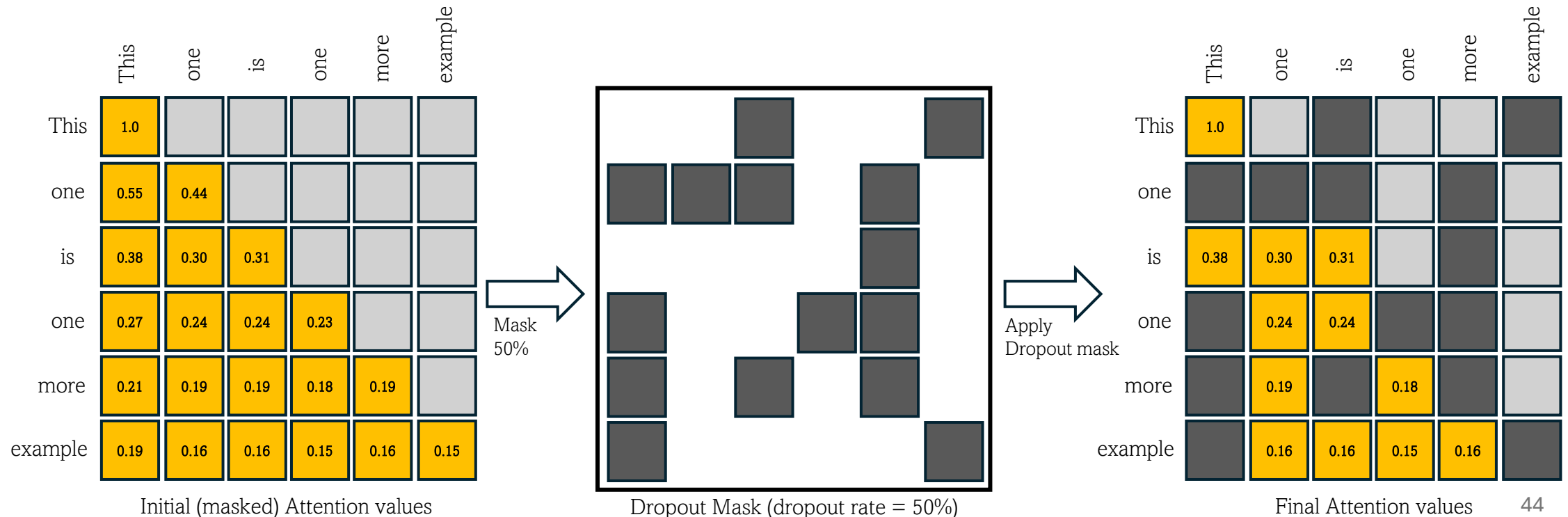


A classic example of Dropout application in an MLP: The solid circles represent the remaining active neurons that were not "dropped out"

GPT: GENERATIVE PRETRAINED TRANSFORMER

DROPOUT

- Dropout is applied randomly on the elements of the **Masked Attention matrix**
 - An additional Dropout mask is used to **zero out random Attention weights**.
 - This randomness can prevent the Attention mechanism from focusing too heavily on particular positions within the input sequence, spreading the "attention" more evenly.



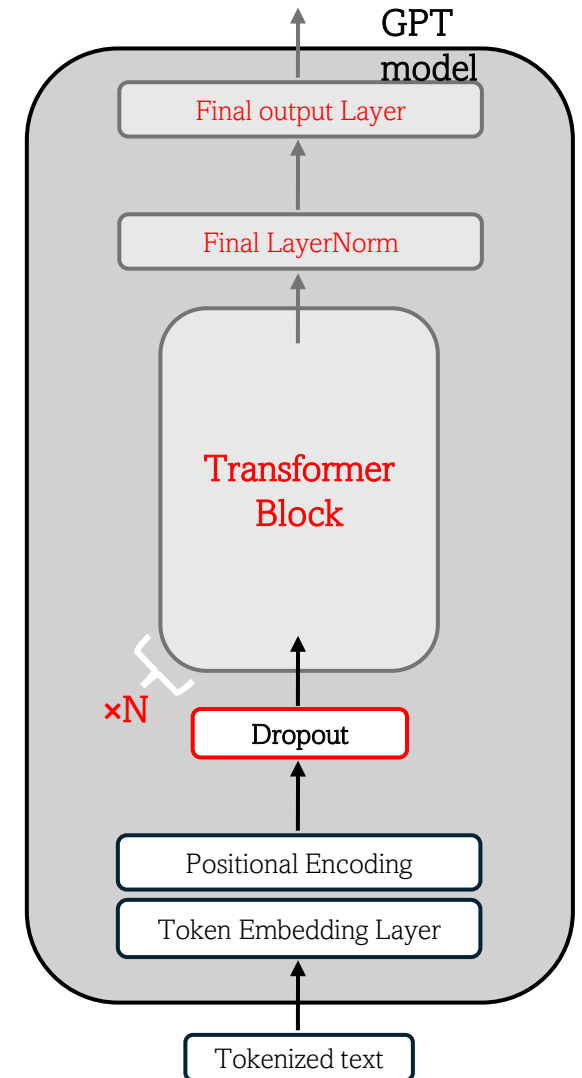
GPT: GENERATIVE PRETRAINED TRANSFORMER

DROPOUT

- Dropout is also **applied to the input embeddings**
 - The model is forced to learn from the input data using a reduced set of features at each training step.
 - The model cannot rely excessively on any particular element of the embeddings for making predictions. Instead, it has to develop a more robust understanding.
- Dropout is **applied independently to every element** of the embedding **with a certain probability**.



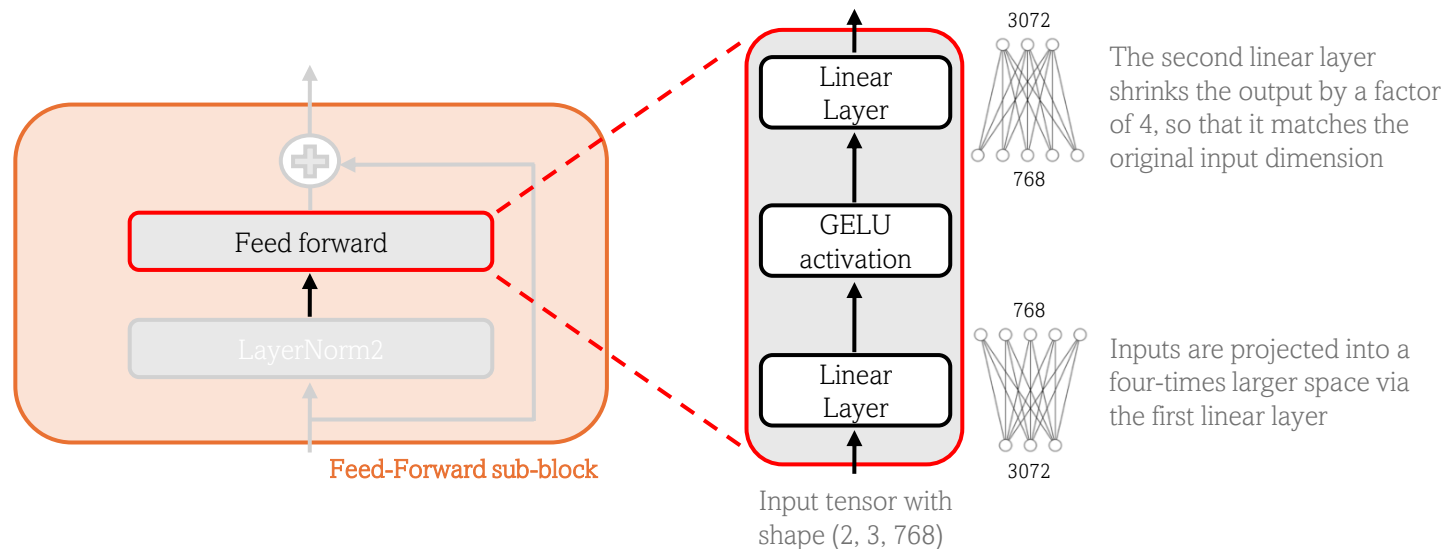
An example of Dropout applied to a 10-dimensional input embedding with probability 0.2



GPT: GENERATIVE PRETRAINED TRANSFORMER

FEED FORWARD MODULE

- The **Feed Forward** module **expands the embedding** dimension into a higher-dimensional space through the first linear layer. Then, a second linear layer **contracts back to the original dimension**.
 - By temporarily projecting into a higher-dimensional space, the model can capture more expressive representations and then compact them back.



GELU combines ideas from ReLU and sigmoid functions, but instead of a hard cutoff (like ReLU's zero for negatives), it weights inputs by their value *and* their probability of being positive under a Gaussian distribution.

$$\text{GELU}(x) \approx 0.5x \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right] \right)$$

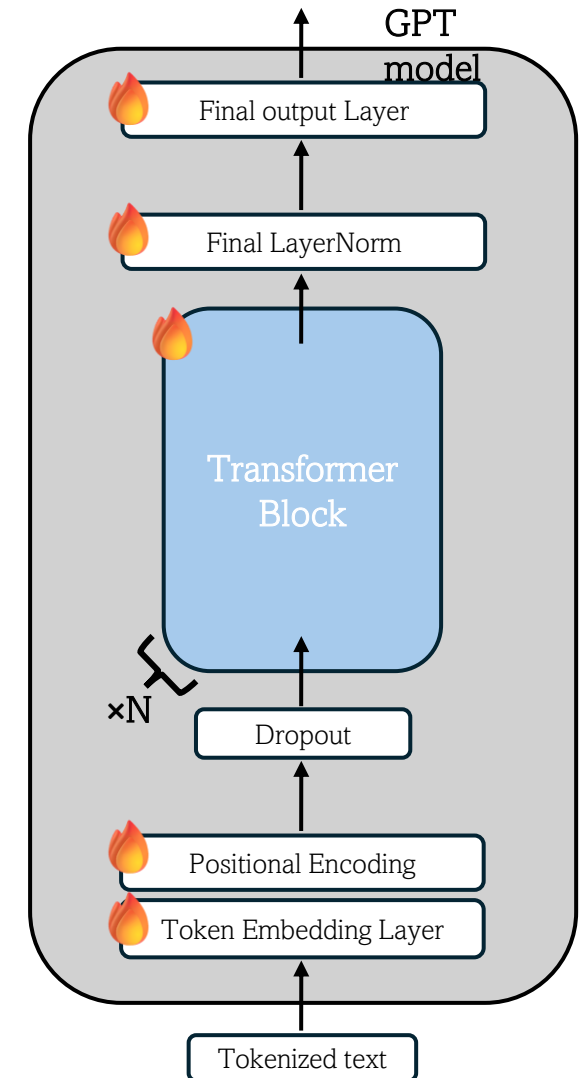
GPT: GENERATIVE PRETRAINED TRANSFORMER

CONCLUSION – TRAINING PROCEDURE

- At its core, **GPT is a stack of Transformer blocks** designed to model language in an **autoregressive** manner, i.e., predicting the next token in a sequence given all previous ones.
- GPT is trained using the **Next-Token Prediction** objective
 - Components marked with 🔥 contain **trainable parameters**, i.e., values that are updated during backpropagation.
 - Training involves maximizing the likelihood of the next (correct) token, or equivalently, **minimizing** its **negative log-likelihood**

$$\mathcal{L}(\mathcal{U}) = - \sum_{i=1}^N \log (P(u_i \mid u_{i-k}, \dots, u_{i-1}, \Theta))$$

Conditional probability of i^{th} token given k preceding tokens and **GPT model parameters Θ**





CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE- TRAINING

CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

INTRODUCTION

- Deep Learning **classification models** are usually **trained** in a supervised manner over a **large, annotated dataset** (e.g., ImageNet [13])
 - However, obtaining large **annotated datasets** is **expensive** as it requires many hours from annotators.
- Is it possible to **learn classification models from data that is already publicly available?**
- **CLIP** [14] is based on the idea that **image–text pairs can be leveraged** as they are readily **accessible on the internet**.
 - How does CLIP use image-text pairs?



A common bottlenose dolphin
(*Tursiops truncatus*)



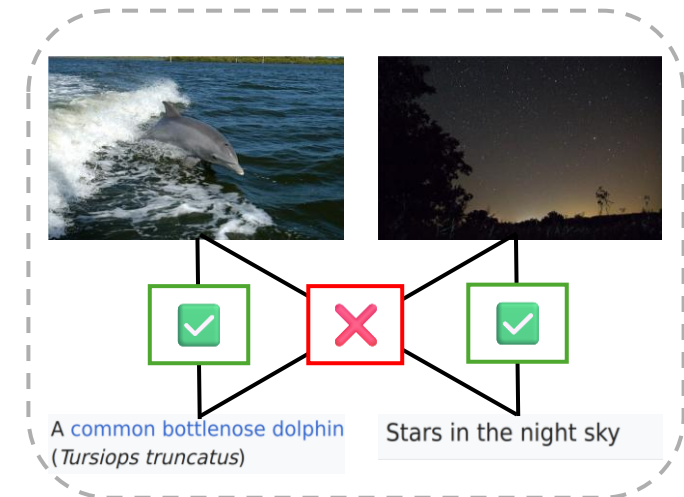
Stars in the night sky

Image-text pairs available
from Wikipedia

CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

INTRODUCTION

- CLIP trains a **visual-linguistic model** over **pairs of images and textual descriptions**.
 - Goal: Building a vision model that can perform different tasks **without task-specific training** or **labeling**.
- CLIP learns visual concepts from image captions by **training to identify** which **image–caption pairs** are **valid**.
 - *This forces the model to associate specific visual features with the words and phrases that describe them.*

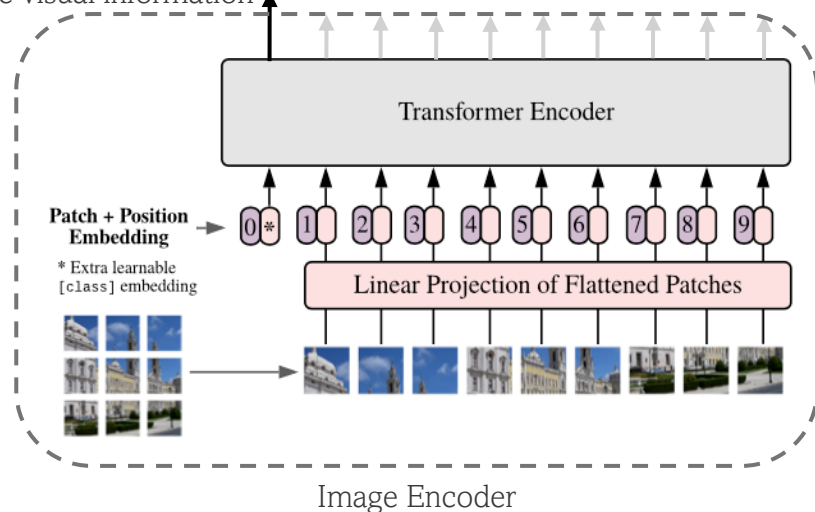


From matched examples (✓), CLIP learns what visual features correspond to words like “dolphin” or “night sky.”
Mismatched pairs (✗) help the model avoid learning incorrect associations.

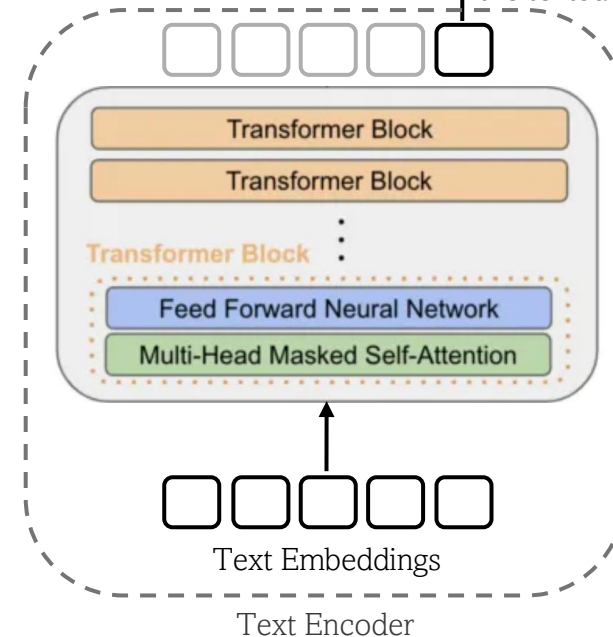
CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING ARCHITECTURE

- **CLIP** is composed of **two encoder modules** that are used to encode textual and image data, respectively.
 - **Image encoder:** ResNet or ViT
 - **Text Encoder:** Decoder-only, typically a Transformer

Only the **first** token is retained to summarize the visual information



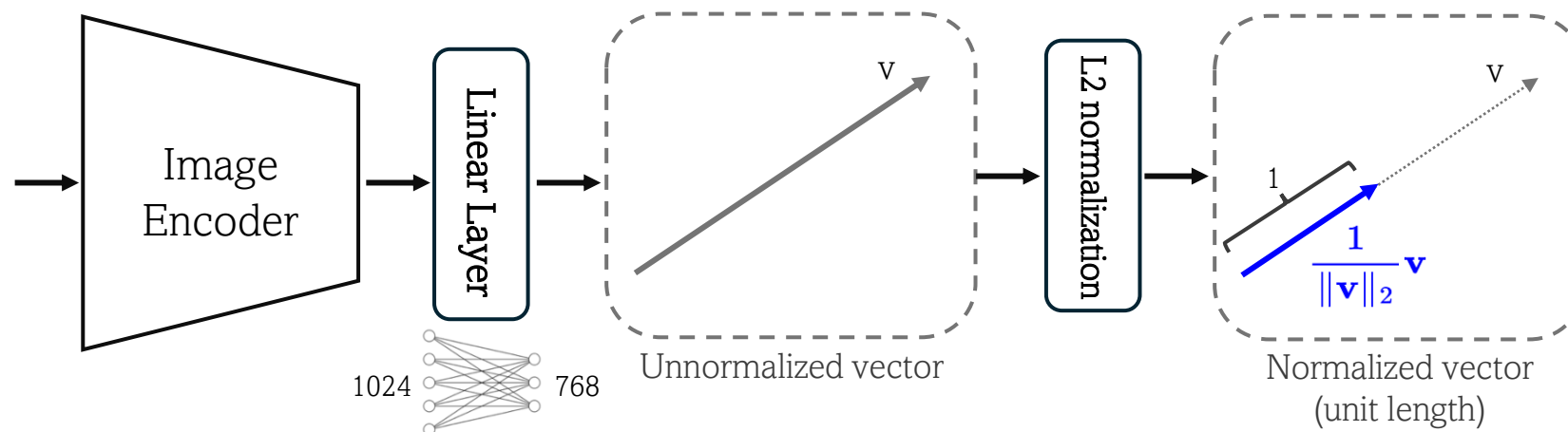
Only the **last** token is retained to summarize the textual information



CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

MEASURE OF SIMILARITY

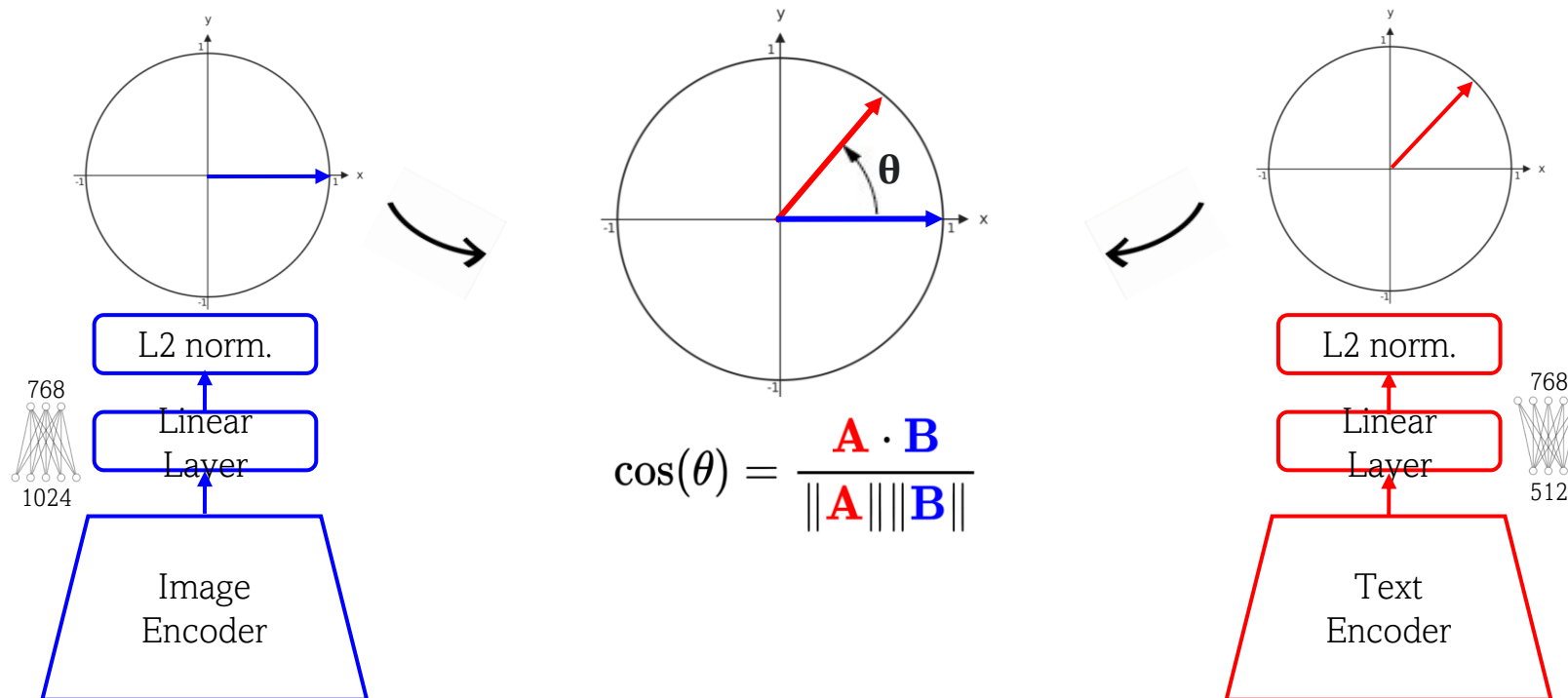
- To identify image-text matches, we need a way to **measure similarity between image and text encoder outputs**.
 - Both image and text outputs are **projected into a space of equal dimensions** to make them directly comparable, using two different **Linear Layers**.
 - **L2 normalization** is applied to **remove scale differences** between the two embeddings, so their similarity can be computed based on how the two point in the same direction (i.e., how well they align), and not on their magnitude.



CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

MEASURE OF SIMILARITY

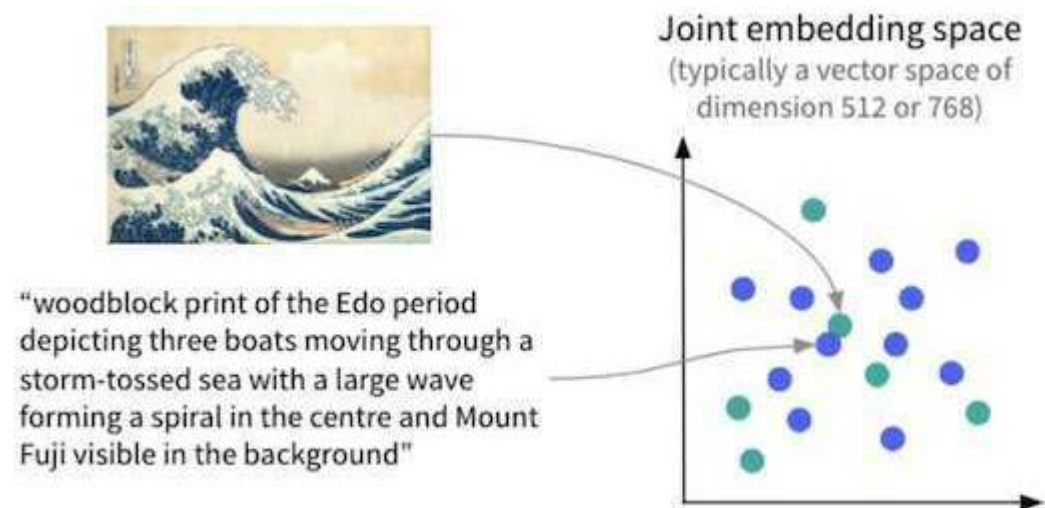
- CLIP measures **image-text embeddings' similarity** by **cosine similarity**.
 - Cosine similarity measures the cosine of the angle between two vectors, telling **how aligned the two vectors are** in terms of directions.
 - For unit vectors, cosine similarity simplifies to just their dot product.



CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

TRAINING OBJECTIVE

- The CLIP training objective consists of **maximizing** the **cosine similarity** between **valid image-text pairs** while **minimizing** the cosine similarity for **all other pairs**.
 - This training objective encourages the formation of a **joint embedding space** for images and text, i.e., image and text encoders are encouraged to **output similar embeddings** for images and textual captions that are related.

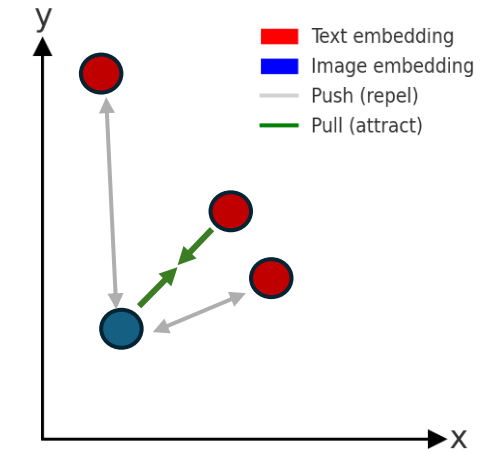


CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

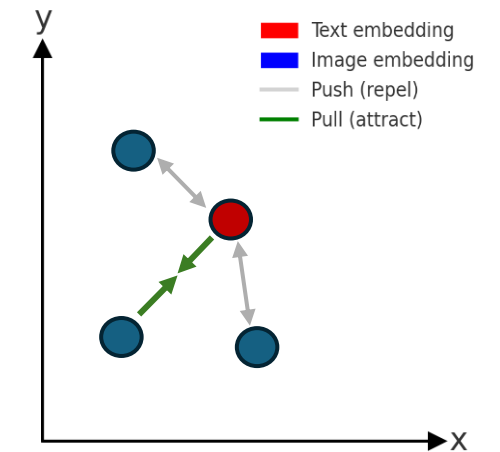
CONTRASTIVE LOSS

- It "contrasts" **positive pairs** (i.e., matching pairs) **with negative pairs** (i.e., non-matching pairs). I.e.,
 - Pulling a matched image and caption closer (in embedding space)
 - Pushing all other combinations (in the batch) apart
- CLIP contrastive loss has **two symmetrical components**:
 - **Image-to-text loss**: Encourage each image to match its paired caption and push apart the rest (in the embedding space)
 - **Text-to-image loss**: Encourage each caption to match its paired image and push apart the rest (in the embedding space)

Image-to-text contrastive loss

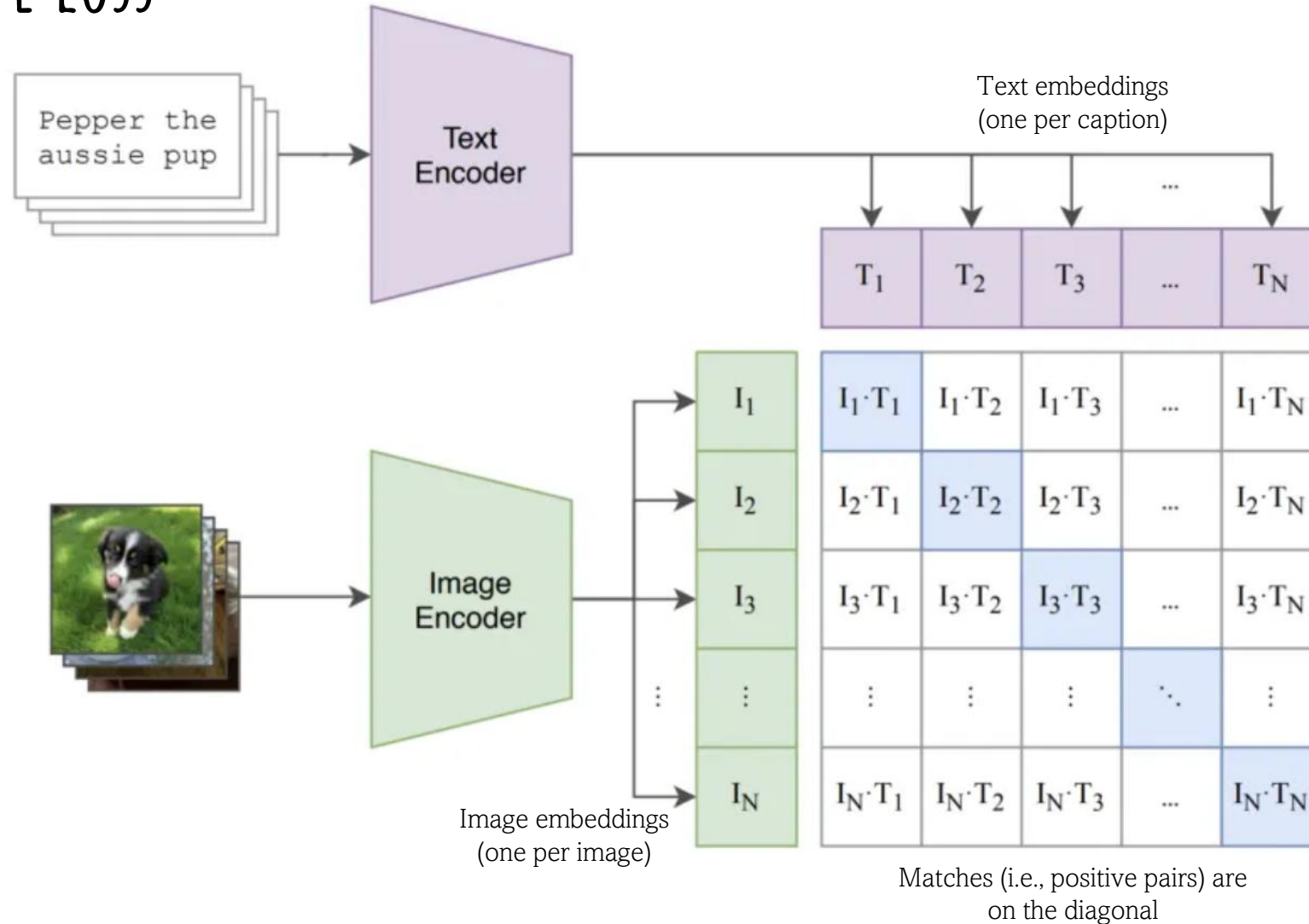


Text-to-image contrastive loss



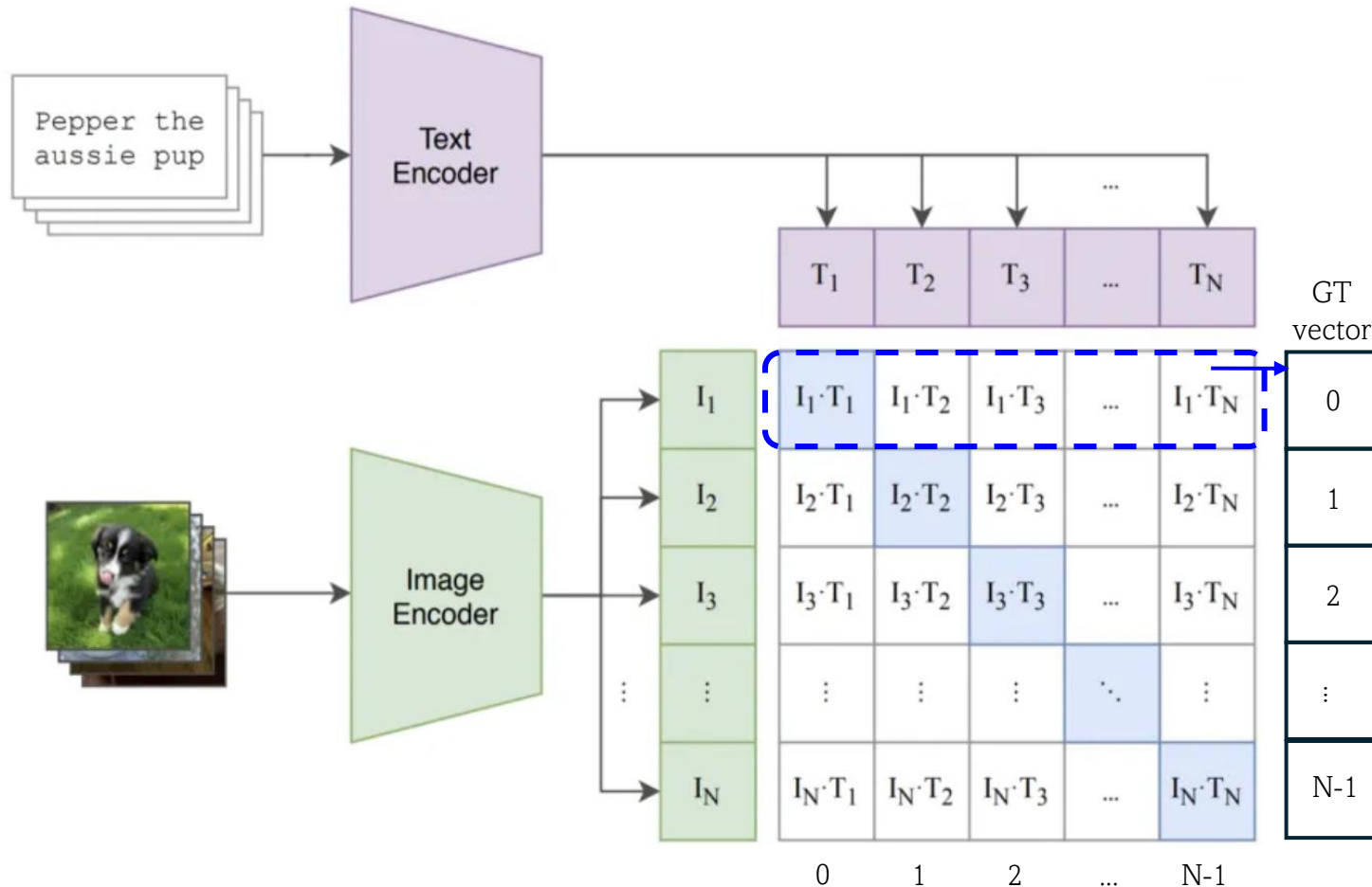
CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

CONTRASTIVE LOSS



CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

IMAGE-TO-TEXT CONTRASTIVE LOSS



- In the image-to-text contrastive loss, each image embedding in a batch is compared against all text embeddings from the same batch.

This results in a set of similarity scores between the image and every caption.

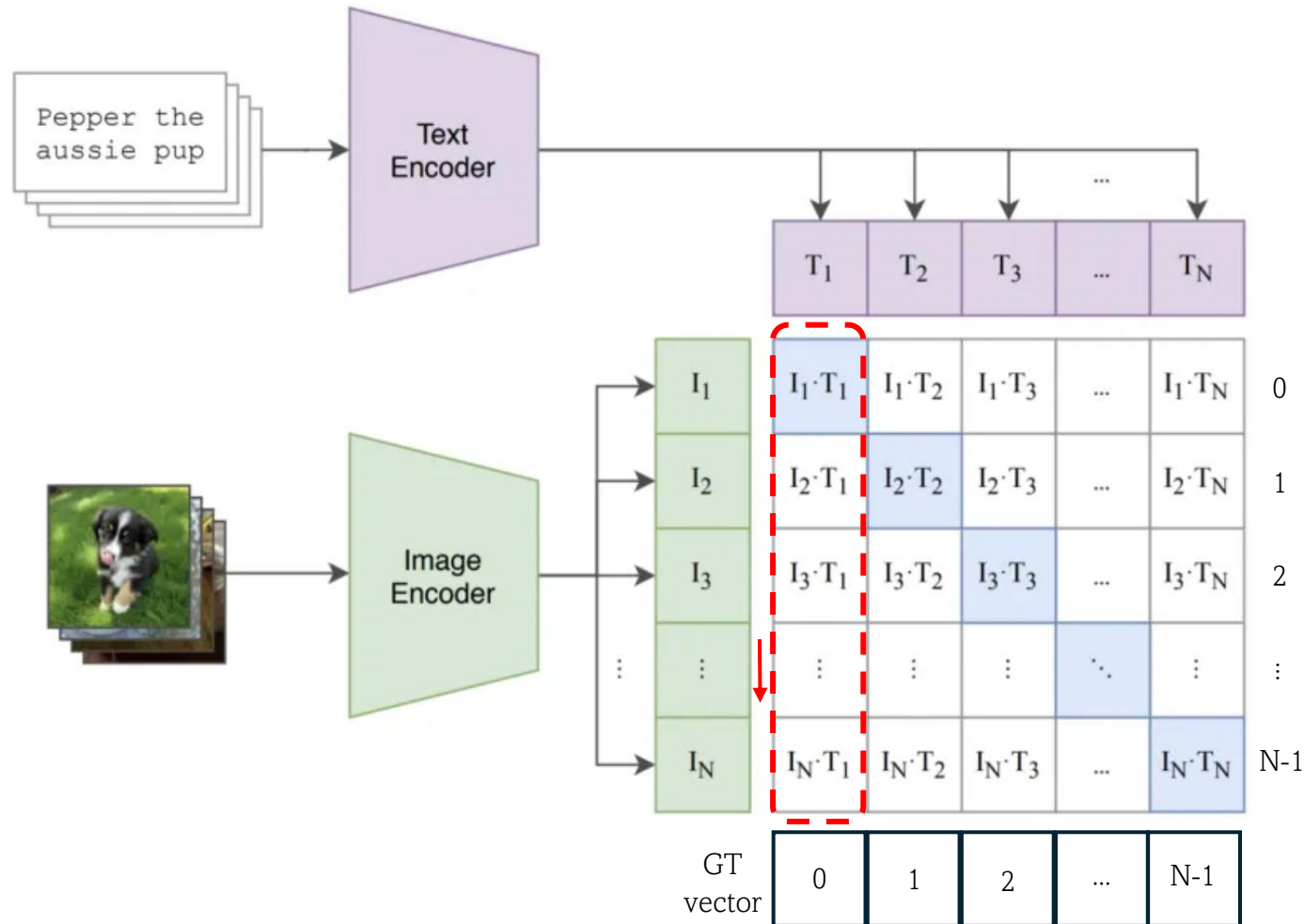
- The ground truth is built as a vector that encodes the position of each true image-text pair, one correct match per image.

- A **Cross-Entropy loss** is applied over the similarities along the text dimension (i.e., on the column axis).

- This encourages the model to assign the **highest similarity score** to the **correct match**, while assigning lower scores to all other captions in the batch.

CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

TEXT-TO-IMAGE CONTRASTIVE LOSS



- In the text-to-image contrastive loss, each text embedding in a batch is compared against all image embeddings from the same batch.

This results in a set of **similarity scores** between the text and every image.

- The **ground truth** is built as a **vector** that encodes the position of each true text-image pair, one correct match per text.

- A **Cross-Entropy loss** is applied over the similarities **along the image dimension** (i.e., on the row axis).

- This encourages the model to assign the **highest similarity score** to the **correct match**, while assigning lower scores of all other images in the batch.

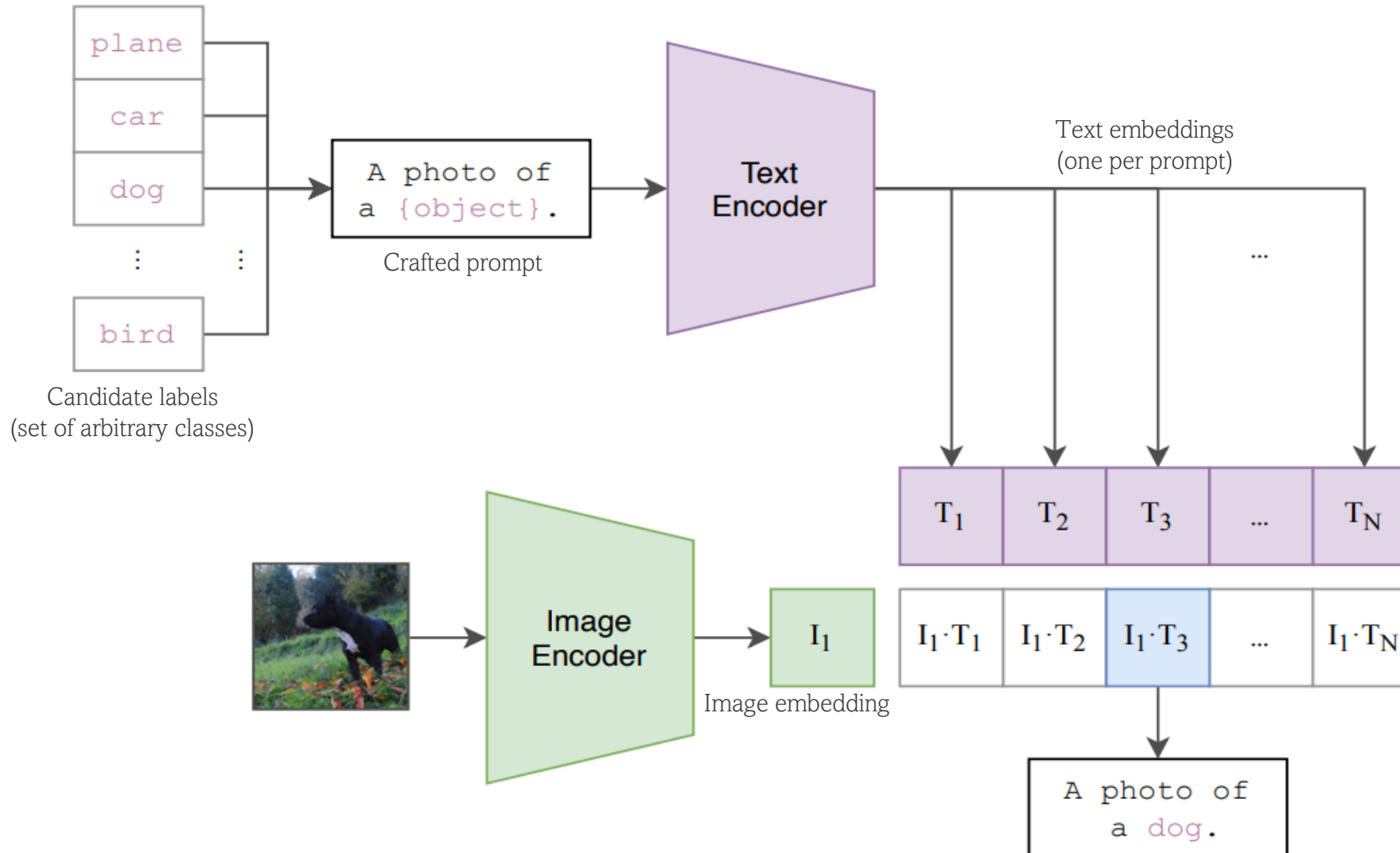
CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

ZERO-SHOT CLASSIFICATION

- CLIP is trained to **predict if an image and a caption are paired**: this capability can be **repurposed** to perform **zero-shot classification**.
- By **crafting textual descriptions for a set of arbitrary classes** (in the form of "**a photo of a <class_name>**"), we can find the most appropriate class for an image by:
 - Using the CLIP text encoder to embed each potential class
 - Using the CLIP image encoder to embed the image
 - Comparing these embeddings to find the image-text pair that is most fitting

CLIP: CONTRASTIVE LANGUAGE-IMAGE PRE-TRAINING

ZERO-SHOT CLASSIFICATION



CLIP performs zero-shot classification by comparing the image embedding with multiple text embeddings derived from candidate labels.

The **prediction** corresponds to the text **prompt** with the **highest cosine similarity**.

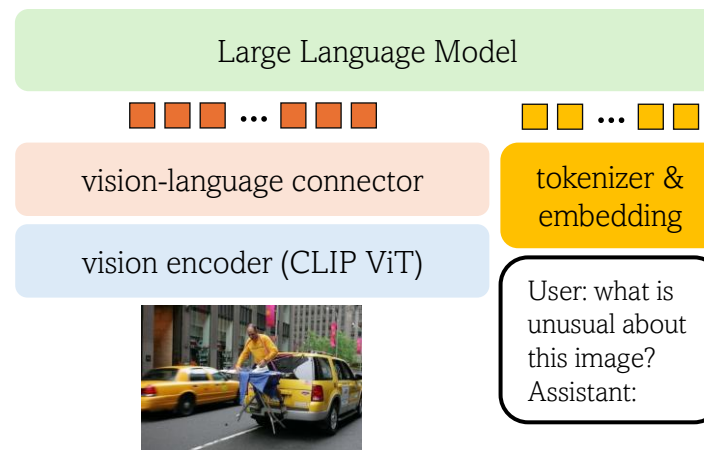


LLAVA: LARGE LANGUAGE AND VISION ASSISTANT

LLaVA: LARGE LANGUAGE AND VISION ASSISTANT

OVERVIEW

- LLaVA [1] is **Vision-Language model**: it works with both **image and text modalities**, integrating both vision and language understanding
 - LLaVA belongs to the **Unified Embedding-Decoder paradigm**
 - It can be used for **generating text responses conditioned on visual inputs**, making it effective for tasks such as **Image Captioning, VQA, and Multimodal, multi-turn conversation**
 - It **builds upon Pre-trained Large Language Models**, such as LLaMA, and Vicuna, and incorporates a vision encoder, typically based on CLIP ViT, to handle image inputs.



LLaVA: LARGE LANGUAGE AND VISION ASSISTANT

MULTIMODAL CHATBOT EXAMPLE



User



User

Do you know who drew this painting?



LLaVA

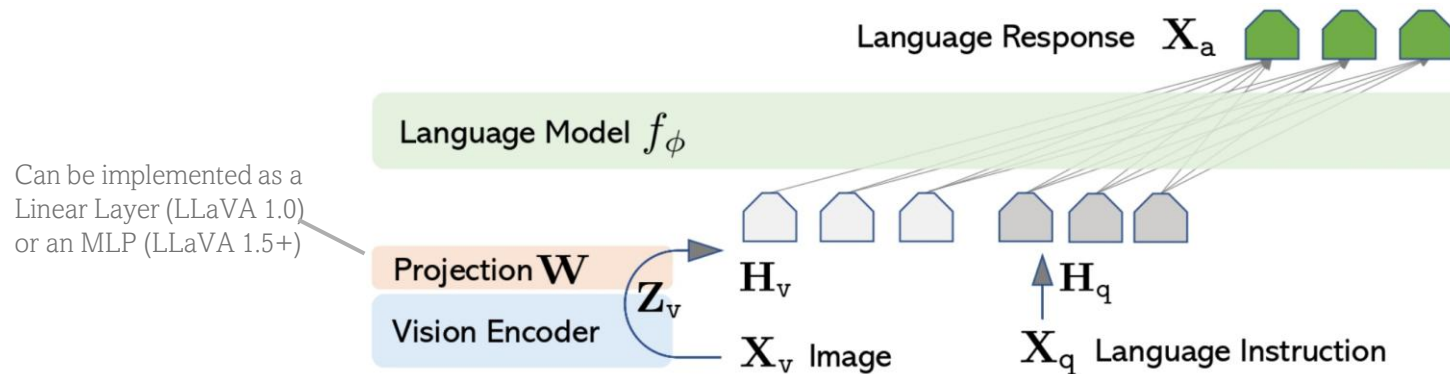
The painting depicts a woman, commonly believed to be Mona Lisa, the famous artwork by Leonardo da Vinci. It is a portrait painting that showcases the woman's enigmatic smile and has become one of the most famous and iconic art pieces in the world. The original work is displayed in the Louvre Museum in Paris, and it is known for its intricate details, use of oil paint, and the artist's innovative techniques that contributed to its enduring appeal and mystery.

Example of a conversation with LLaVA used as a Multimodal Chatbot

LLaVA: LARGE LANGUAGE AND VISION ASSISTANT

ARCHITECTURE SUMMARY

- LLaVA's design relies on three key components: **Vision Encoder**, **Language Model**, and **Multimodal Projection**
 - The Vision Encoder extracts **visual features**, the LLM processes and generates **language**, and the projector aligns the two modalities

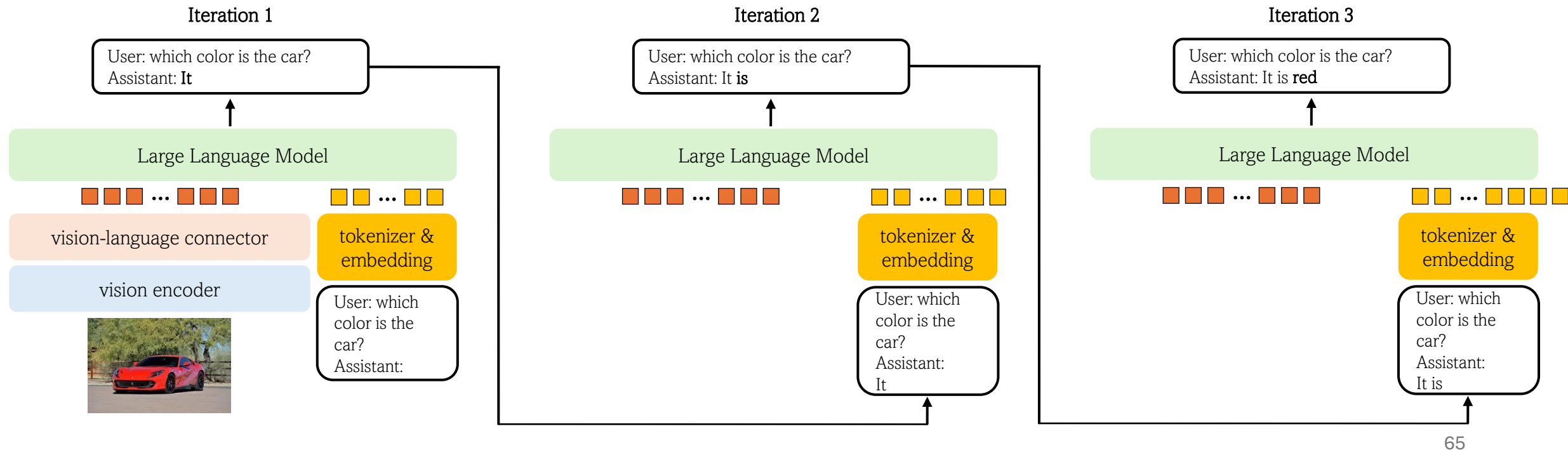


- The **core of LLaVA's** strength lies in its ability to **align and integrate visual and textual modalities**
- The **projector reprocesses the visual embeddings** into a format that the language model can interpret as if they were text tokens, bridging the gap between image and text

LLaVA: LARGE LANGUAGE AND VISION ASSISTANT

AUTOREGRESSIVE MODELING

- LLaVA follows the **autoregressive** modeling paradigm
 - Each new token/word is chosen based on the sequence that precedes it
 - The **visual input and question establish the context**, which the model extends token by token



LLaVA: LARGE LANGUAGE AND VISION ASSISTANT

TRAINING PROCEDURE

- LLaVA is trained using the **Next-Token Prediction** objective
 - Training involves maximizing the likelihood of the next (correct) token, or equivalently, **minimizing** its negative **log-likelihood**, conditioned on the image, instruction, and prior tokens

$$\mathcal{L}_{\text{LLaVA}} = - \sum_{i=1}^L \log P(u_i \mid \mathbf{X}_v, \mathbf{X}_{\text{instruct}}, \mathbf{X}_{a,<i}, \Theta)$$

The diagram illustrates the components of the LLaVA loss function. Below the equation, four labels are connected to their respective terms in the formula by lines:

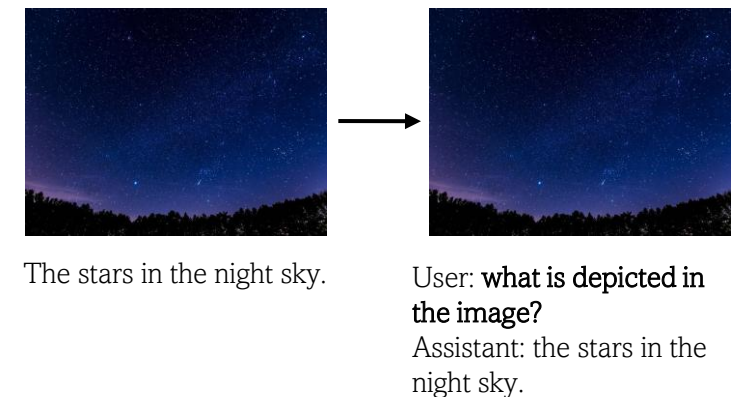
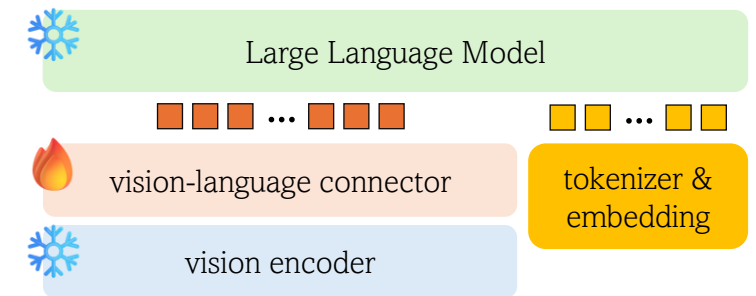
- u_i : The i^{th} token in the assistant's answer (target token to predict)
- $\mathbf{X}_v$: Visual input
- $\mathbf{X}_{\text{instruct}}$: User's input instruction
- $\mathbf{X}_{a,<i}$: All previously generated answer tokens before position i

- Training is divided into two stages:
 - Pre-training
 - Fine-tuning

LLAVA: LARGE LANGUAGE AND VISION ASSISTANT

PRE-TRAINING FOR FEATURE ALIGNMENT

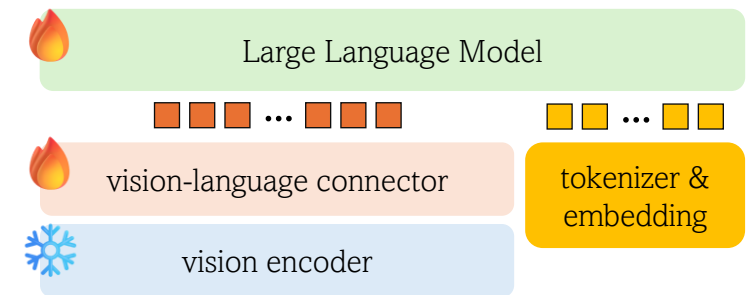
- The goal of the Pre-training stage is to **align the outputs of the vision encoder** with the input embeddings of the language model
 - This alignment is necessary as the vision encoder (e.g., CLIP ViT) and language model (e.g., Vicuna, LLaMA) are pre-trained **separately** and thus operate in **incompatible** embedding spaces
 - **Only the parameters of the projection layer**, which maps visual features to the language model's token space, **are updated** (🔥)
- A diverse set of **image-text pairs** (e.g., CC3M dataset, 3M image-caption pairs) is **used for training** to ensure a broad concept coverage
 - Textual captions are converted into **instruction-following data** (**single-turn conversation**), where the input is a question related to the image (e.g., "What is depicted in the image?")



LLaVA: LARGE LANGUAGE AND VISION ASSISTANT

FINE-TUNING (VISUAL INSTRUCTION TUNING)

- The **Visual Instruction Tuning** stage **fine-tunes the pre-trained model** to handle complex tasks that require multimodal understanding and reasoning, and **to act as an assistant**
 - While pre-training gives LLaVA the ability to understand visual information broadly, fine-tuning teaches it how to use that information effectively in conversations
 - During fine-tuning, the **LLM parameters are updated jointly with the projection layer** (🔥) to support multimodal instruction tuning
- In this stage, the model is trained on **multi-turn conversations**
 - Only after being trained on this kind of data, the model will **behave like a chatbot**: remembering context across turns and continuing to answer a series of questions conversationally



User: what meals can I cook with these?
Assistant: with the variety of food items stored in the refrigerator, you can prepare several meals. For example ...
User: I'd like fruit salad. Show me the recipe.
Assistant: Certainly! Here's a simple fruit salad recipe using the items in your refrigerator ...

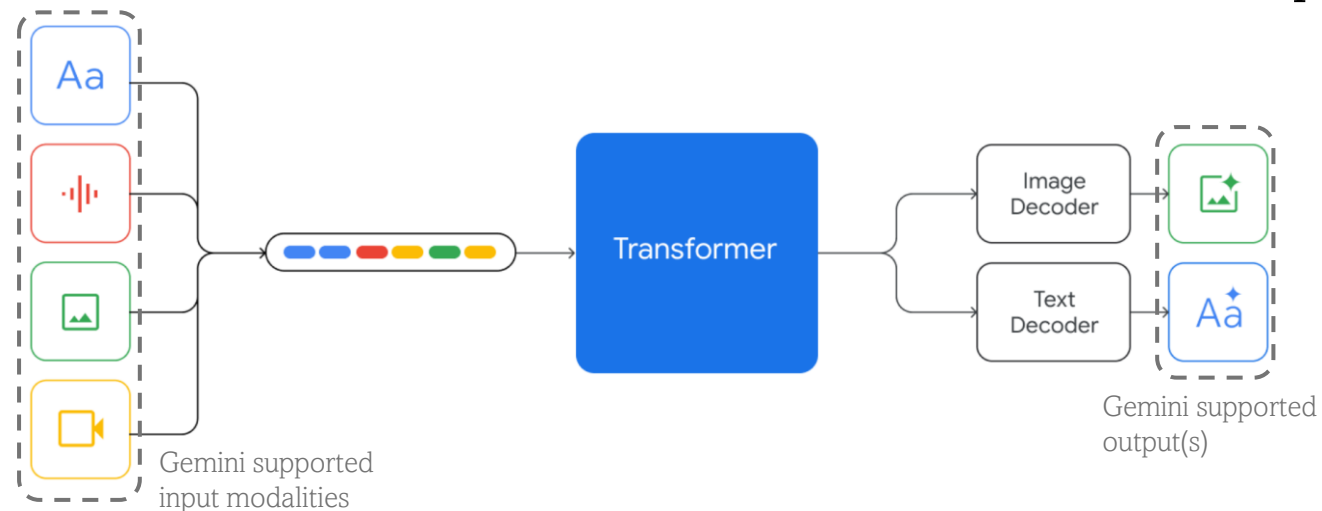


GEMINI: A FAMILY OF HIGHLY CAPABLE MULTIMODAL MODELS

GEMINI: A FAMILY OF HIGHLY CAPABLE MULTIMODAL MODELS

INTRODUCTION

- **Gemini** [15] is a family of **Multimodal AI models** designed to process multiple modalities, including **audio**, **images**, **text**, and **video**
- Unlike GPT models discussed in this lesson, which accept only text-based prompts as input, **Gemini supports interleaved sequences of different modalities as inputs** and **can produce interleaved text and image as outputs**
 - The model doesn't need all the text first and then the images — it can handle them mixed in a natural conversation-like flow. The same for producing a reply.



GEMINI: A FAMILY OF HIGHLY CAPABLE MULTIMODAL MODELS

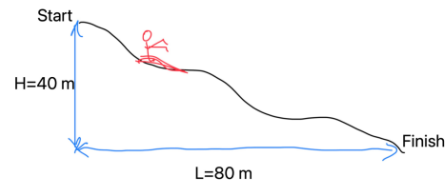
EXAMPLE



Prompt

Here's a solution to a Physics problem by a student.

23. A skier slides down the frictionless slope as shown. What is the skier's speed at the bottom?



Solution:

The conservation of energy means that the energy of the skier is same at the beginning and at the end.

Start: $E = mgL$

End: $E = \frac{1}{2}mv^2$

$$\Rightarrow mgL = \frac{1}{2}mv^2$$

$$\Rightarrow v = \sqrt{2gL} = \sqrt{2 \times 9.81 \times 80} = 39.6 \text{ m.s}^{-1}$$

Try to reason about the question step by step.

Did the student get the correct answer? If the solution is wrong, please explain what is wrong and solve the problem.

Make sure to use LaTeX for math and round off the final answer to two decimal places.



Response

The student did not get the correct answer. The student made a mistake in the calculation of the potential energy at the start of the slope. The potential energy at the start of the slope is given by mgh , where m is the mass of the skier, g is the acceleration due to gravity, and h is the height of the slope. The student used the length of the slope, L , instead of the height, H , in the calculation of the potential energy.

The correct solution is:

The conservation of energy means that the total energy of the skier is the same at the beginning and at the end of the slope.

At the start of the slope, the skier has only potential energy, which is given by mgh .

At the end of the slope, the skier has only kinetic energy, which is given by $\frac{1}{2}mv^2$

Therefore, we can write:

$$mgh = \frac{1}{2}mv^2$$

Canceling out the mass m on both sides, we get:

$$gh = \frac{1}{2}v^2$$

Solving for v , we get:

$$v = \sqrt{2gh}$$

Substituting the given values, we get:

$$v = \sqrt{2 \times 9.81 \times 40} = 28.01 \text{ m/s}$$

Therefore, the skier's speed at the bottom of the slope is 28.01 m/s.

GEMINI: A FAMILY OF HIGHLY CAPABLE MULTIMODAL MODELS

MULTILINGUAL CAPABILITIES

- Gemini is trained on a dataset that is both **multimodal** and **multilingual**
 - Trained on high-quality English text and 40+ other languages

Prompt

The Basic Chinese Family Tree

Explain what i should call my grandparents on each side of my family based on this chart

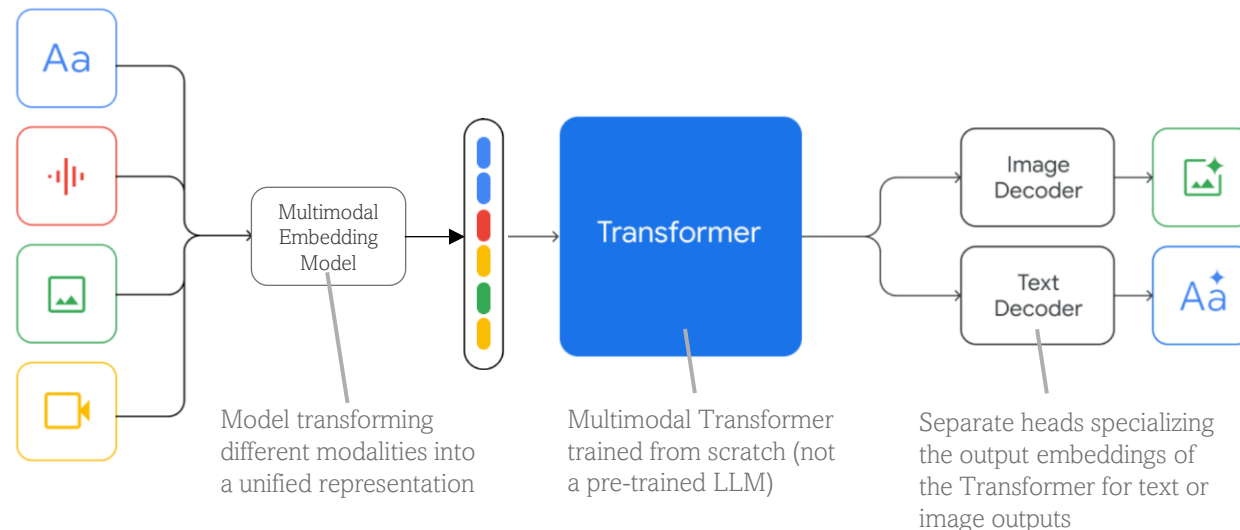
Model Response

On your father's side, you would call your grandparents Yéye and Nǎinai. On your mother's side, you would call your grandparents Wàigōng and Wàipó.

GEMINI: A FAMILY OF HIGHLY CAPABLE MULTIMODAL MODELS

ARCHITECTURE *HYPOTHESIS*

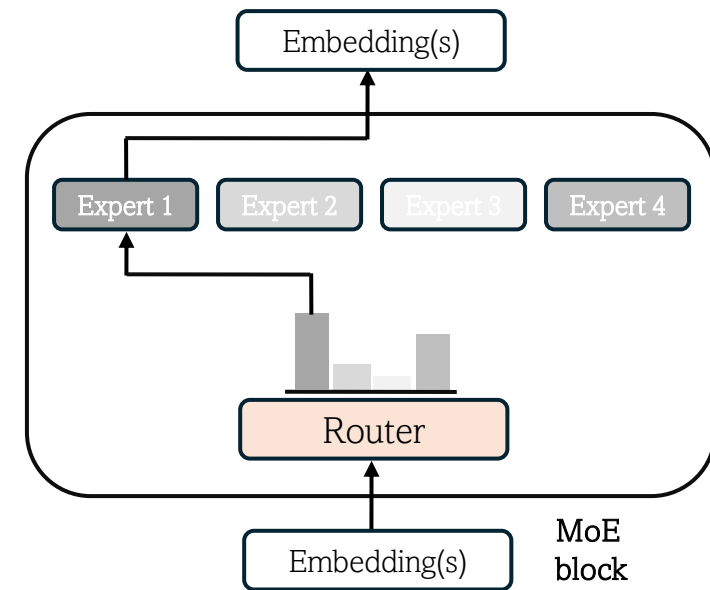
- Although **Gemini architecture details are not fully disclosed**, there are **hints** provided in Gemini technical report [15] that give us an idea of how Gemini *could* be structured
 - *"Gemini models are built on top of Transformer decoders"*
 - *"Gemini models are multimodal from the beginning"*



GEMINI: A FAMILY OF HIGHLY CAPABLE MULTIMODAL MODELS

MIXTURE OF EXPERTS (MOE)

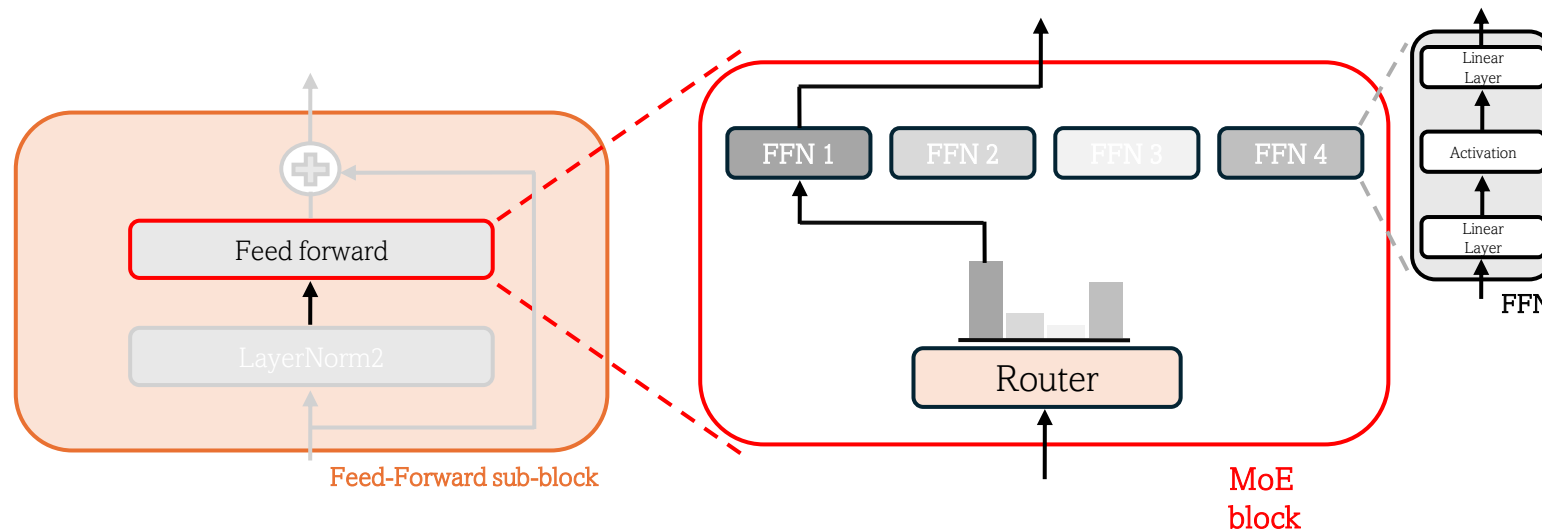
- Gemini 1.5 [16] introduces the **Mixture of Experts (MoE)** technique
 - MoE divides a network into smaller "expert" networks
- MoE uses a **learned routing function** to direct inputs to one or more experts
 - Depending on the type of input given, MoE models learn to selectively activate only the most relevant expert(s) in the network
 - This specialization **enhances model efficiency** as only a subset of the model is active at inference, minimizing total compute (FLOPS)
- Where is MoE applied inside the Transformer?



GEMINI: A FAMILY OF HIGHLY CAPABLE MULTIMODAL MODELS

MIXTURE OF EXPERTS (MOE)

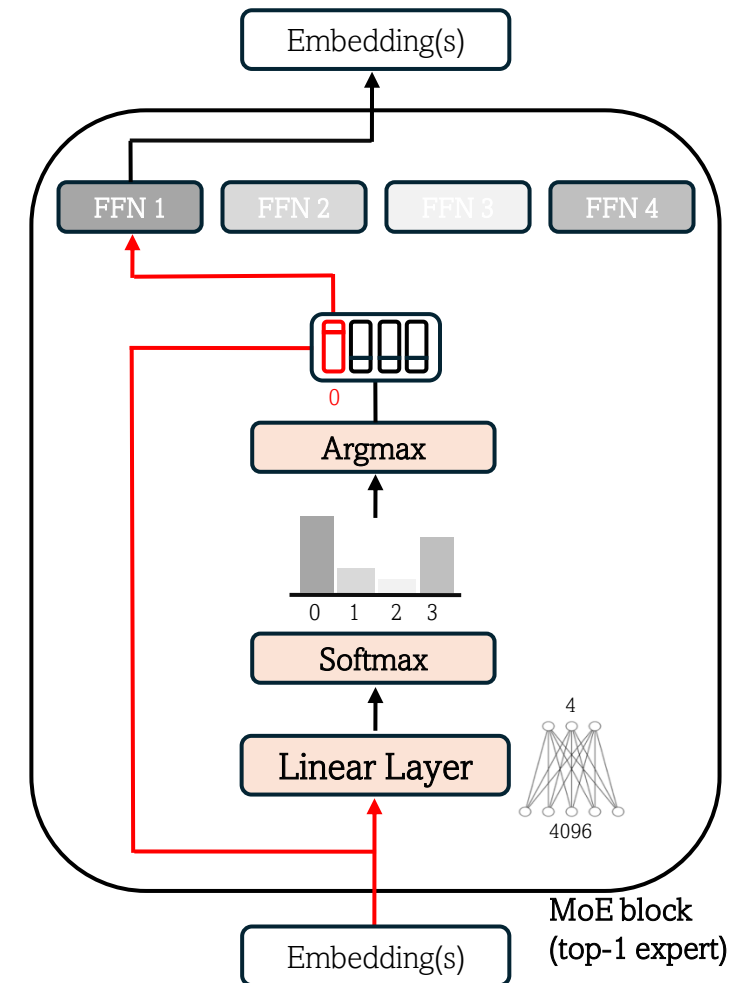
- **MoE can replace the Feed forward modules in the Transformer**
 - Instead of using a single Feed Forward Network (FFN), a MoE block can be used to **direct tokens** to the most relevant expert(s) **based on their content**
 - During training, the **Router** mechanism **learns to route tokens** in a way that encourages each expert to specialize in different input patterns
 - By routing each token to just one or two experts, **MoE avoids running the full model** on every input, which makes it feasible to **include far more parameters than in a dense model**



GEMINI: A FAMILY OF HIGHLY CAPABLE MULTIMODAL MODELS

MIXTURE OF EXPERTS (MOE) - ROUTER

- The **Router** determines which **expert** (or top-k experts) should **process each token**. The Router decides:
 - Which expert(s) should handle each token
 - How much weight to give to each expert's output (if top-k experts are used)
- The **Router** can be implemented as a **Linear Layer** followed by a **Softmax-based selection**
- In the **single expert** configuration:
 - It **projects the embeddings** to a vector of **dimension** equal to the **number of experts**
 - Apply a **Softmax** to get a probability distribution
 - Apply an **Argmax** to select the expert based on the highest probability



Sources & Credits

- <https://cameronrwolfe.substack.com/>
- <https://sebastianraschka.com/blog/>
- <https://www.manning.com/books/build-a-large-language-model-from-scratch>

Bibliography

- [1] Liu, Haotian, et al. "Visual instruction tuning." *Advances in neural information processing systems* 36 (2023).
- [2] Zhu, Deyao, et al. "Minigpt-4: Enhancing vision-language understanding with advanced large language models." *arXiv preprint arXiv:2304.10592* (2023).
- [3] Wang, Peng, et al. "Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution." *arXiv preprint arXiv:2409.12191* (2024).
- [4] Grattafiori, Aaron, et al. "The llama 3 herd of models." *arXiv preprint arXiv:2407.21783* (2024).
- [5] Li, Dongxu, et al. "Aria: An open multimodal native mixture-of-experts model." *arXiv preprint arXiv:2410.05993* (2024).
- [6] Dai, Wenliang, et al. "Nvlm: Open frontier-class multimodal llms." *arXiv preprint arXiv:2409.11402* (2024).

Bibliography

- [7] Dosovitskiy, Alexey, et al. "An image is worth 16x16 words: Transformers for image recognition at scale." arXiv preprint arXiv:2010.11929 (2020).
- [8] Sennrich, Rico, Barry Haddow, and Alexandra Birch. "Neural machine translation of rare words with subword units." *arXiv preprint arXiv:1508.07909* (2015).
- [9] Radford, Alec, et al. "Improving language understanding by generative pre-training." (2018).
- [10] Radford, Alec, et al. "Language models are unsupervised multitask learners." *OpenAI blog* 1.8 (2019): 9.
- [11] Brown, Tom, et al. "Language models are few-shot learners." *Advances in neural information processing systems* 33 (2020): 1877-1901.
- [12] Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from overfitting." *The journal of machine learning research* 15.1 (2014): 1929-1958.

Bibliography

- [13] Deng, Jia, et al. "Imagenet: A large-scale hierarchical image database." *2009 IEEE conference on computer vision and pattern recognition*. Ieee, 2009.
- [14] Radford, Alec, et al. "Learning transferable visual models from natural language supervision." *International conference on machine learning*. PmLR, 2021.
- [15] Team, Gemini, et al. "Gemini: a family of highly capable multimodal models." *arXiv preprint arXiv:2312.11805* (2023).
- [16] Team, Gemini, et al. "Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context." *arXiv preprint arXiv:2403.05530* (2024).
- [17] Liu, Aixin, et al. "Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model." *arXiv preprint arXiv:2405.04434* (2024).



That's all

Cigdem Beyan
cigdem.beyan@univr.it
<https://cbeyan.github.io/>